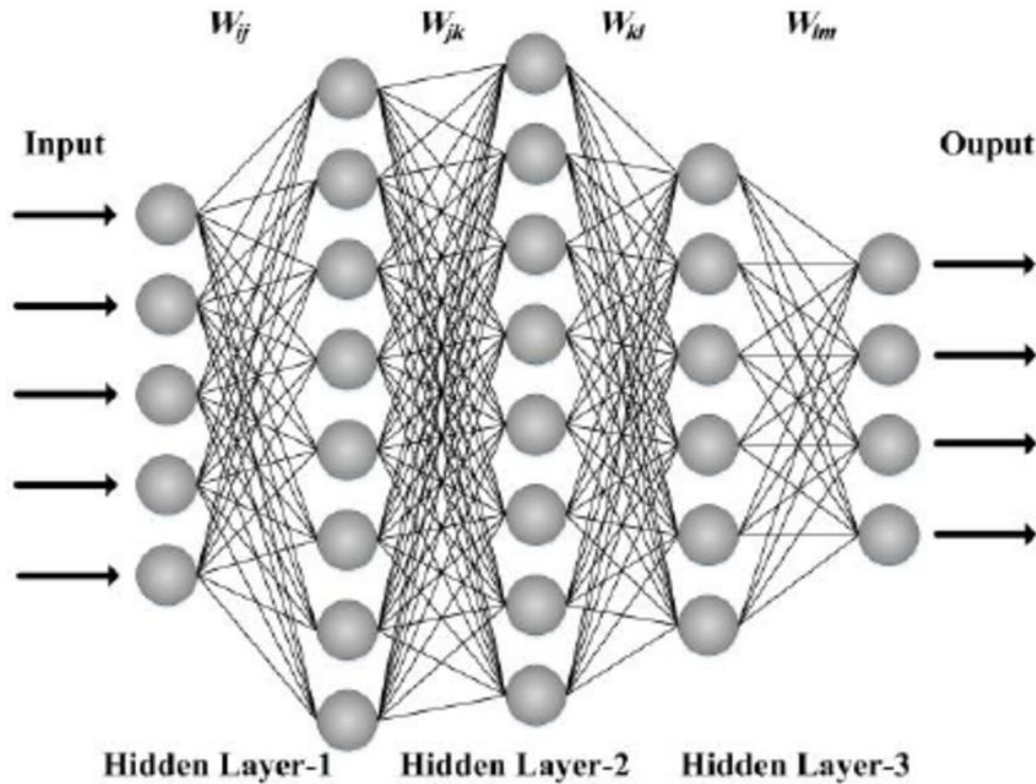
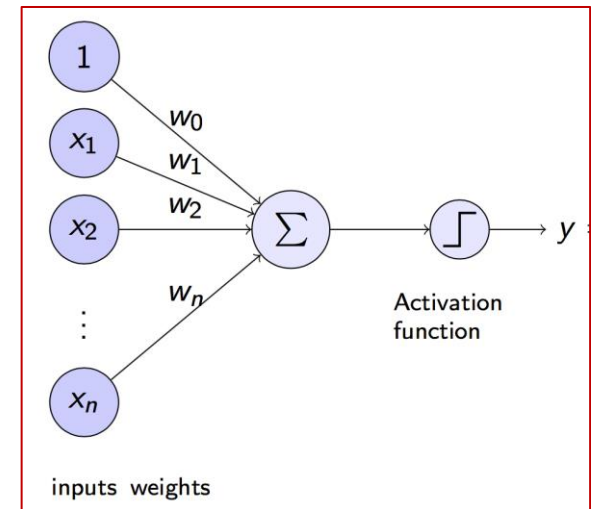
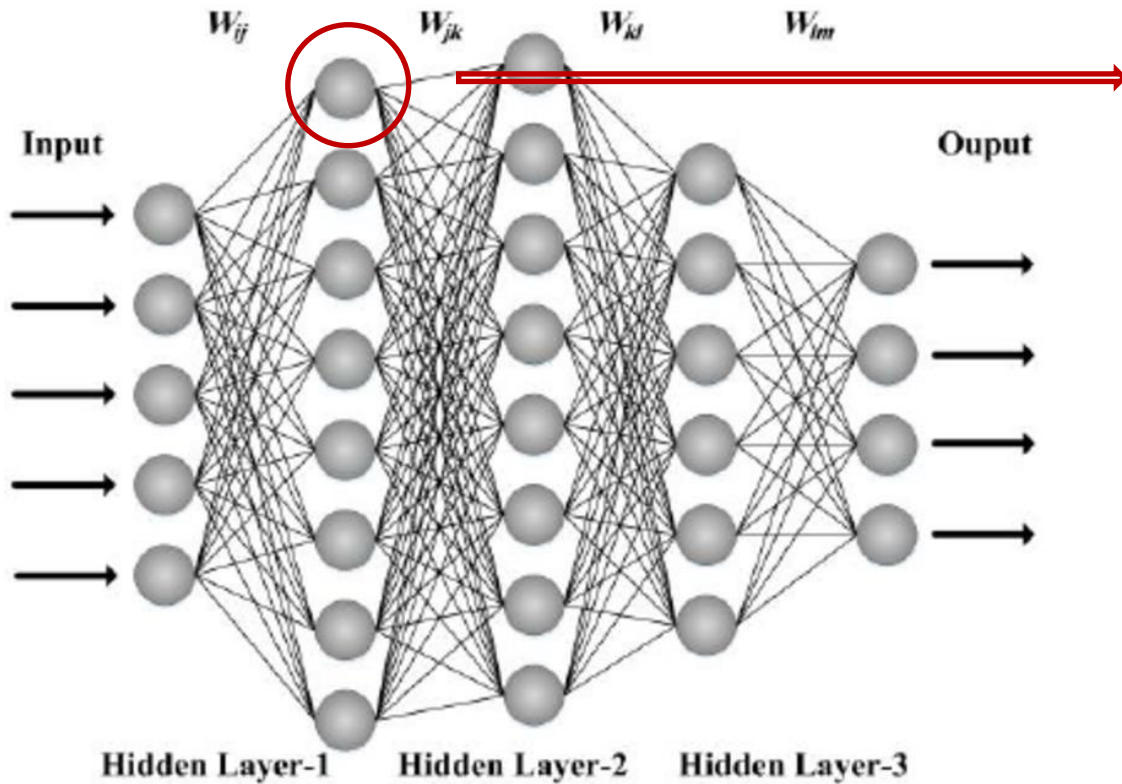


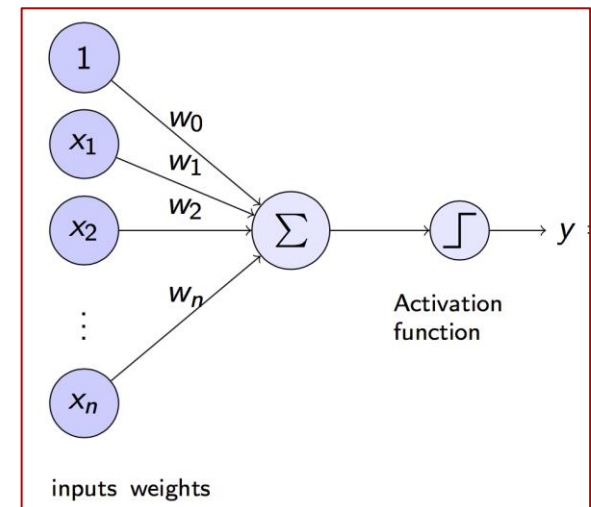
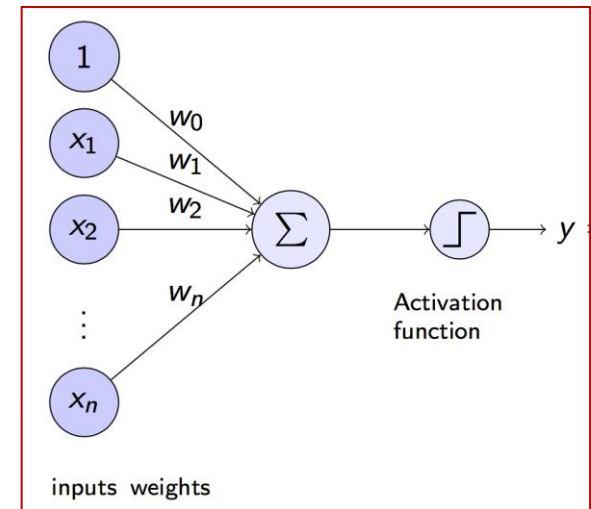
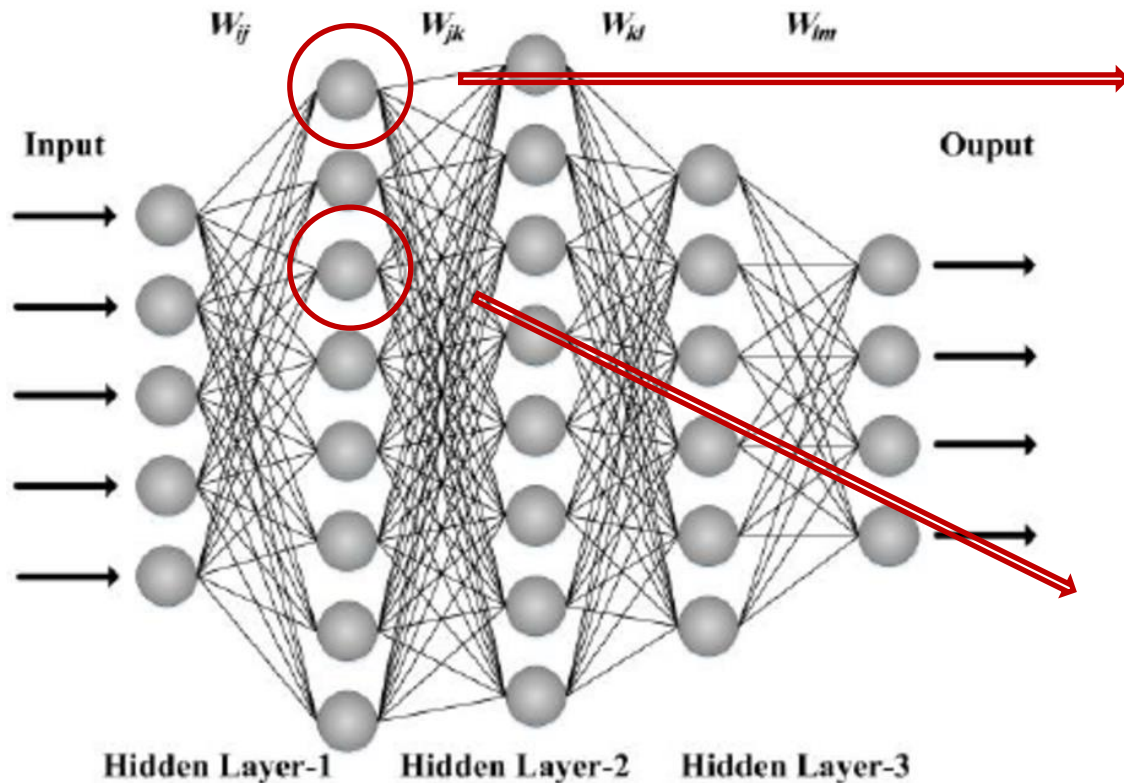
Sharing Weights: Convolutional Networks



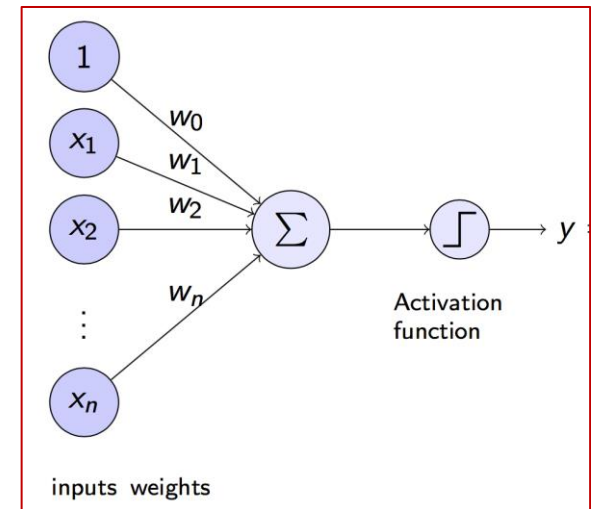
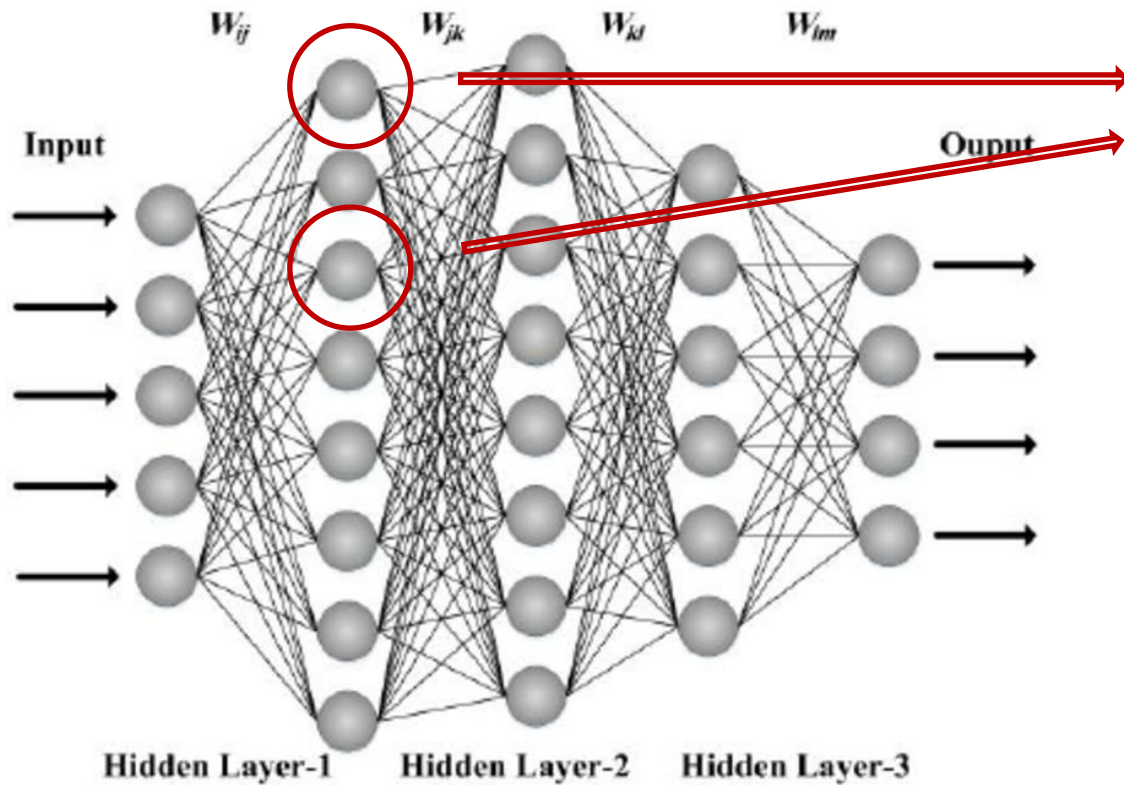
Sharing Weights: Convolutional Networks



Sharing Weights: Convolutional Networks



Sharing Weights: Convolutional Networks



Convolutional Networks

Convolution:

$$s(t) = \int x(a)w(t - a)da, \text{ usually denoted } s(t) = (x * w)(t)$$

Convolutional Networks

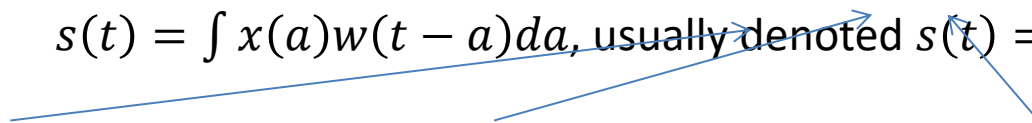
Convolution:

$$s(t) = \int x(a)w(t-a)da, \text{ usually denoted } s(t) = (x * w)(t)$$

Feature map

Input

Kernel



Convolutional Networks

Convolution:

$$s(t) = \int x(a)w(t-a)da, \text{ usually denoted } s(t) = (x * w)(t)$$

Feature map

Input

Kernel

Important notions:

Sparse Interactions

Parameter Sharing

Equivalent Representations

Convolutional Networks

Convolution:

$$s(t) = \int x(a)w(t-a)da, \text{ usually denoted } s(t) = (x * w)(t)$$

Feature map

Input

Kernel

Important notions:

Sparse Interactions: total number of weights required $\ll$ number of inputs

Parameter Sharing

Equivalent Representations

Convolutional Networks

Convolution:

$$s(t) = \int x(a)w(t - a)da, \text{ usually denoted } s(t) = (x * w)(t)$$

The diagram illustrates the convolution equation $s(t) = \int x(a)w(t - a)da$ and its shorthand notation $s(t) = (x * w)(t)$. Three blue arrows originate from labels below the equation: one from 'Feature map' pointing to the integral symbol, one from 'Input' pointing to $x(a)$, and one from 'Kernel' pointing to w in the shorthand notation $(x * w)(t)$.

Important notions:

Sparse Interactions: total number of weights required $\ll$ number of inputs

Parameter Sharing: same weights used for many neurons

Equivalent Representations

Convolutional Networks

Convolution:

$$s(t) = \int x(a)w(t - a)da, \text{ usually denoted } s(t) = (x * w)(t)$$

The diagram illustrates the convolution equation $s(t) = \int x(a)w(t - a)da$ and its shorthand notation $s(t) = (x * w)(t)$. Three blue arrows point from labels below to terms in the equation: one from 'Feature map' to $x(a)$, one from 'Input' to $x(a)$, and one from 'Kernel' to $w(t - a)$.

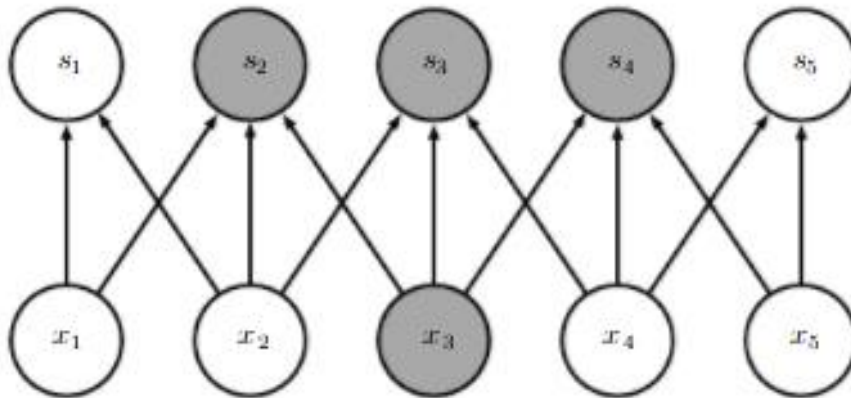
Important notions:

Sparse Interactions: total number of weights required $\ll$ number of inputs

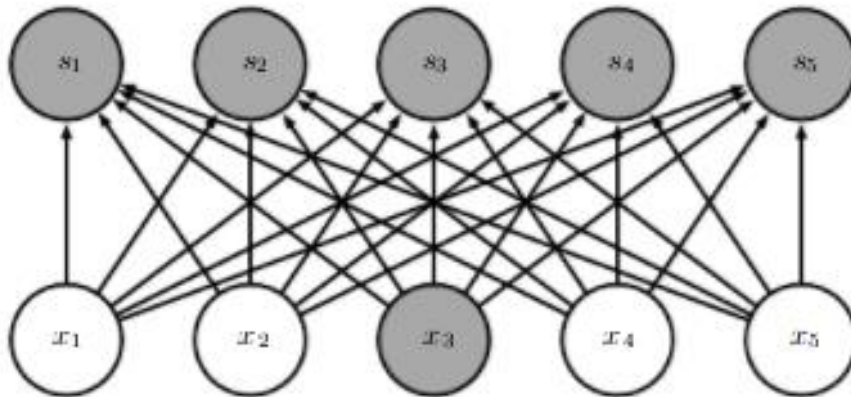
Parameter Sharing: same weights used for many neurons

Equivalent Representations: a transformation to the input is similarly applied to the output

Convolutional Networks

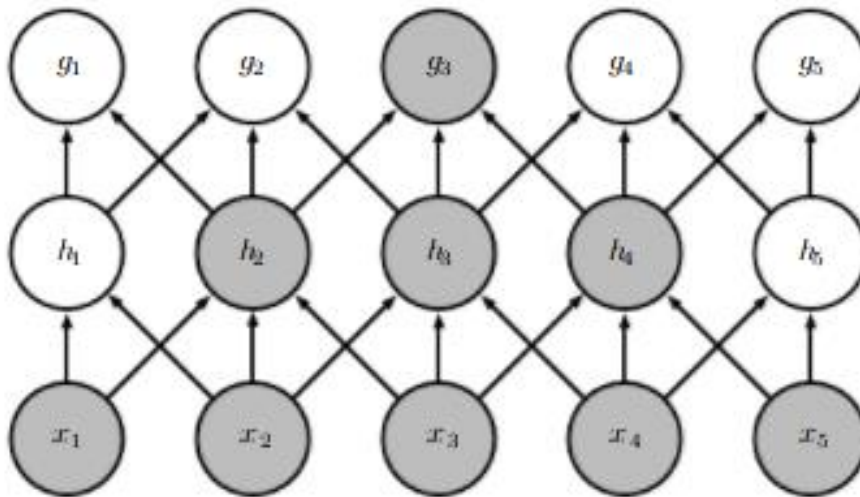


Sparse
Connections



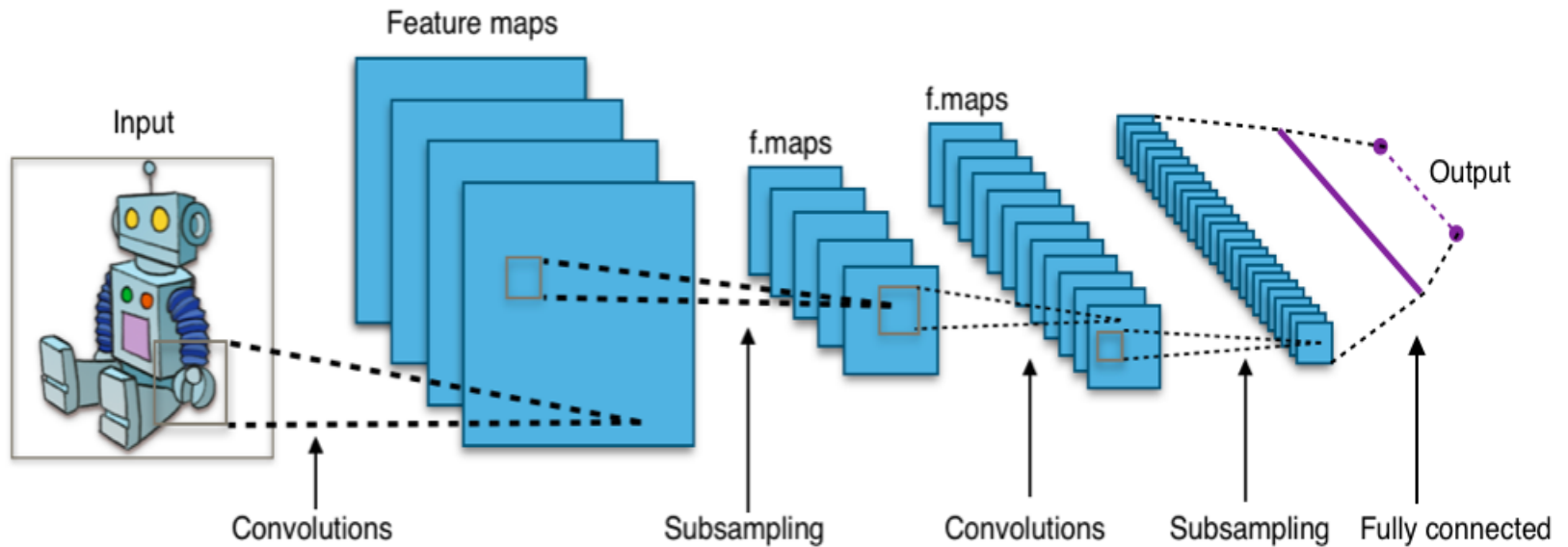
Dense
Connections

Convolutional Networks



Indirect propagation of
information through
sparse connections

Convolutional Networks



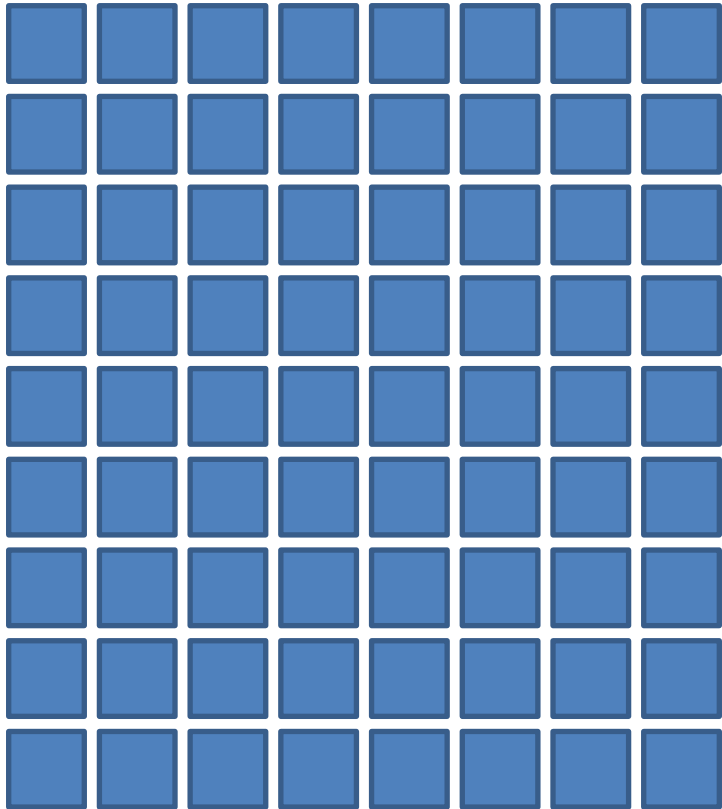
Transform an image into a set
number of feature images:

(here a contour detecting feature)



Convolutional Network

Input

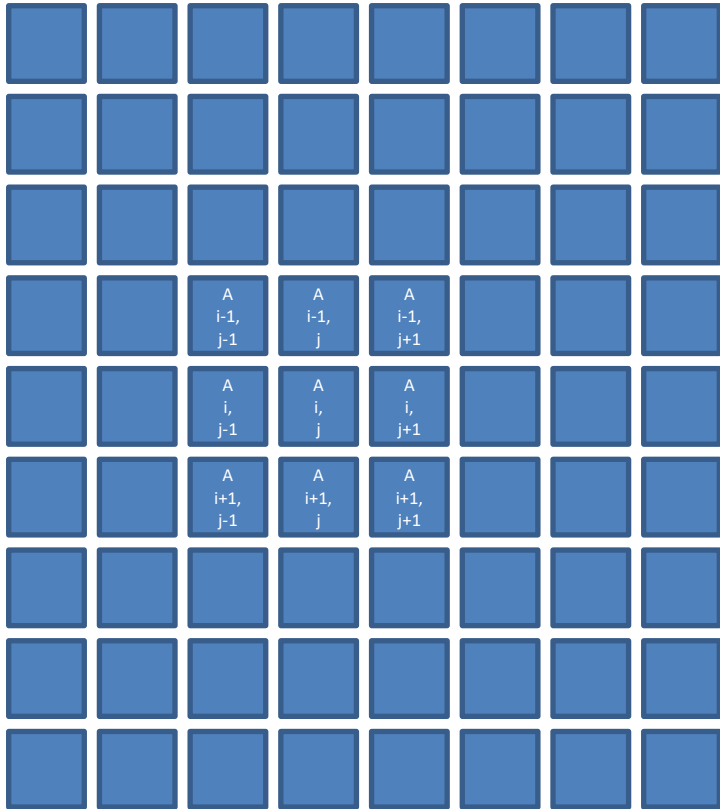


Kernel



Convolutional Network

Input



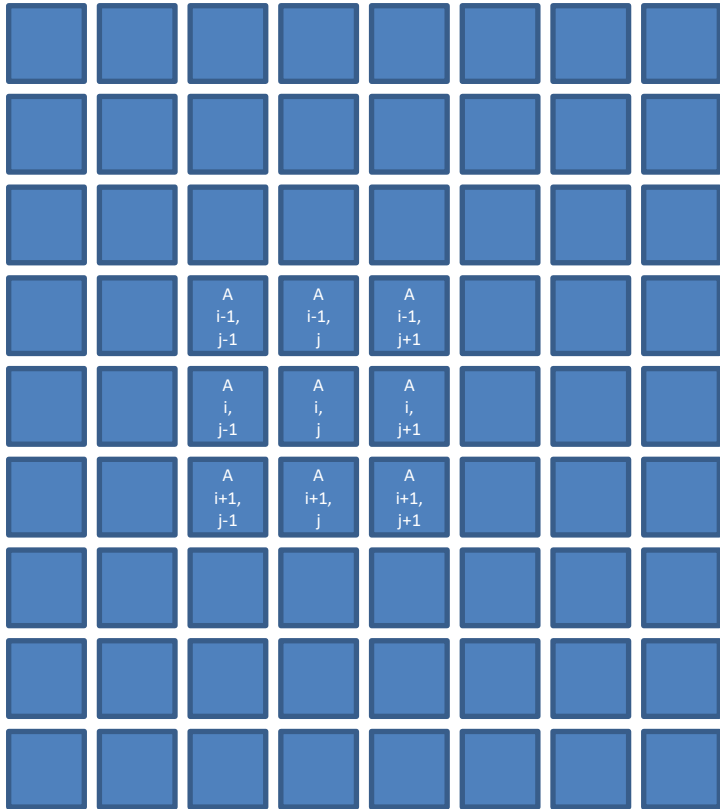
Kernel



$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

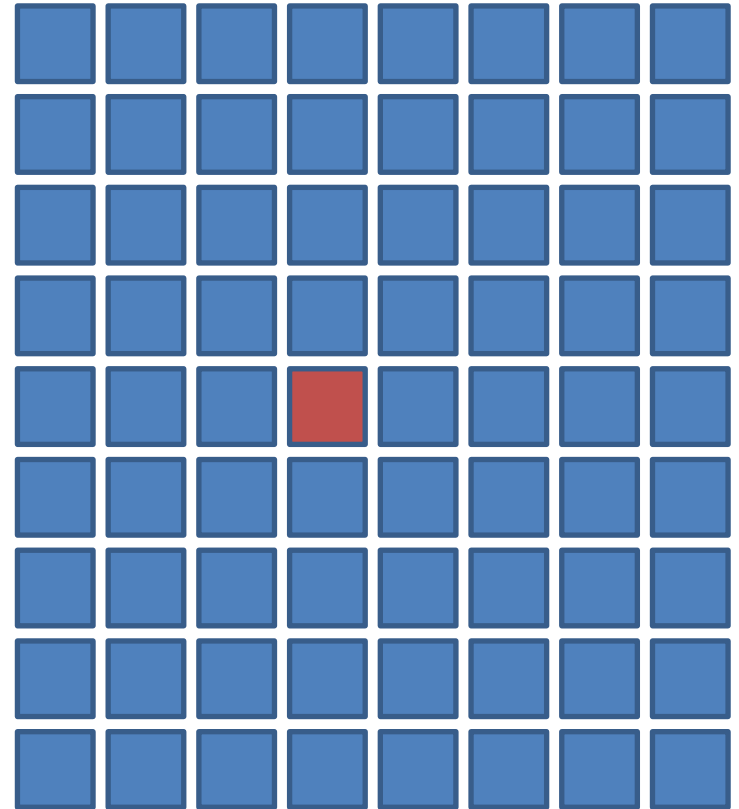
Input



Kernel



Feature Map



$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

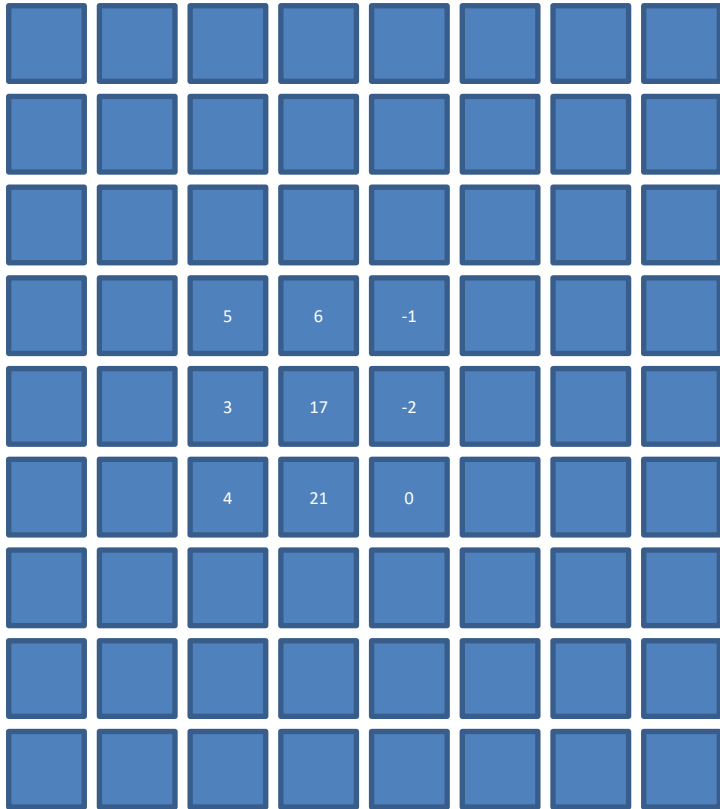
Kernel

0	-1	0
-1	5	-1
0	-1	0

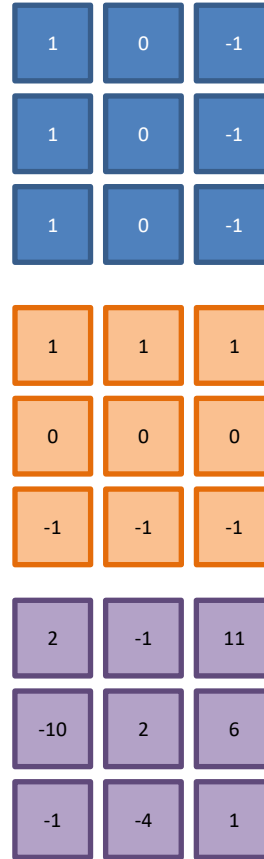
114				

Convolutional Network

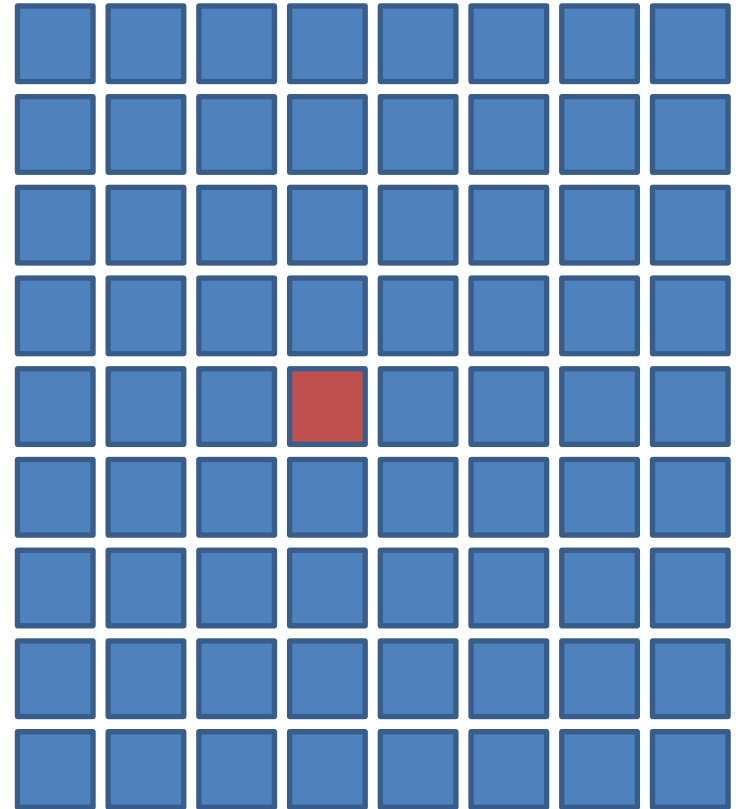
Input



Kernel



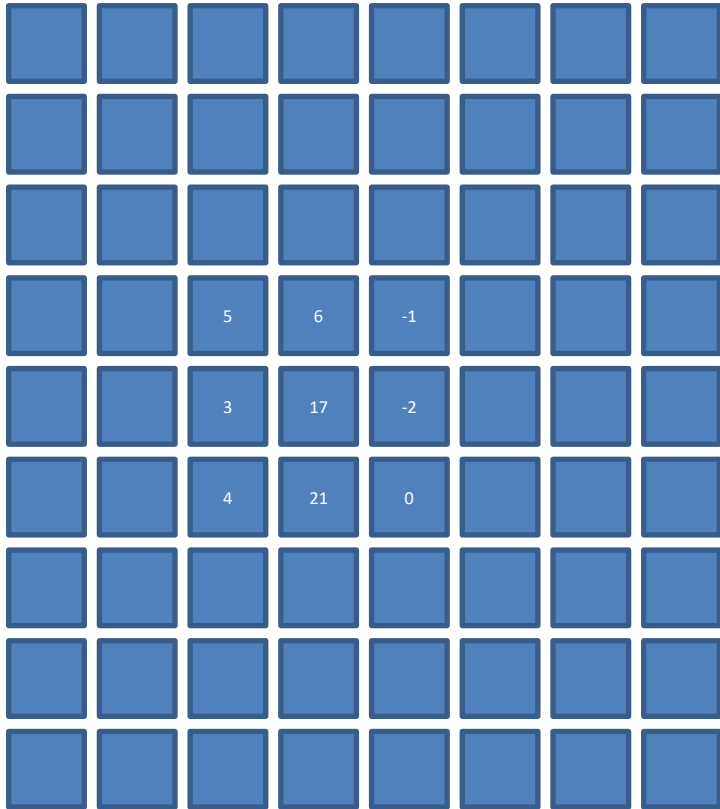
Feature Map



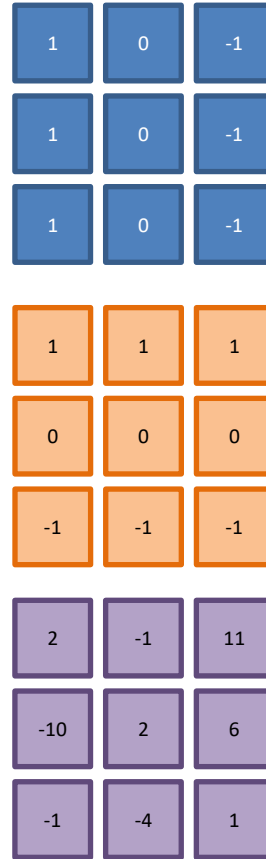
$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

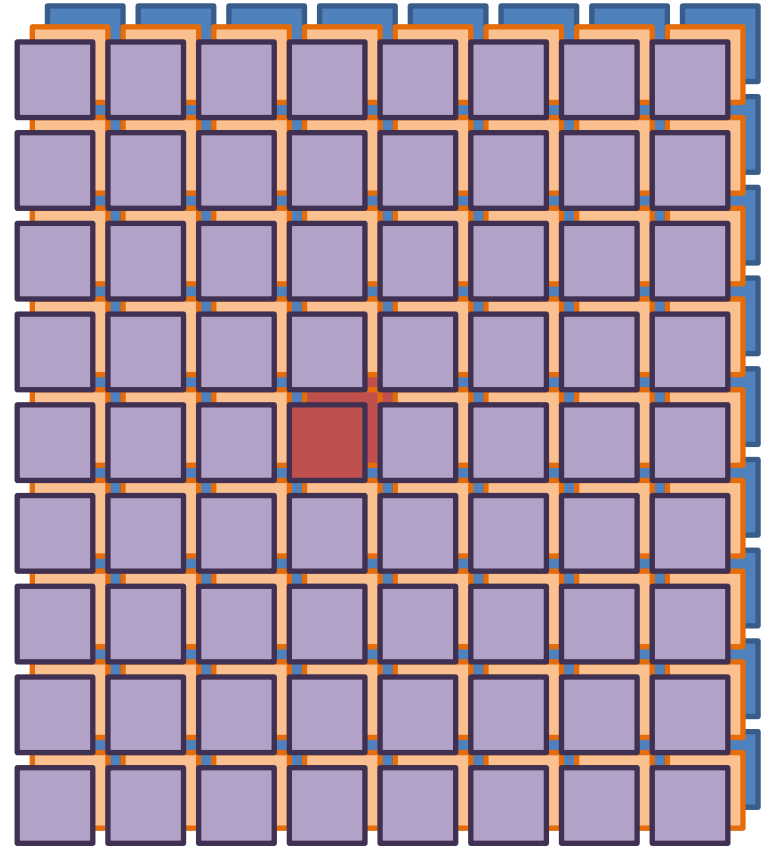
Input



Kernel



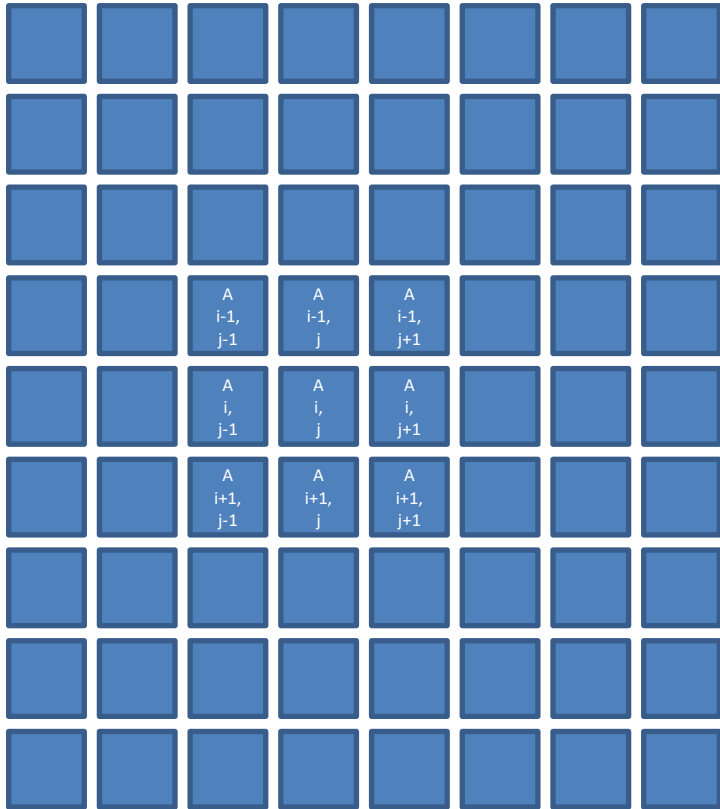
Feature Map



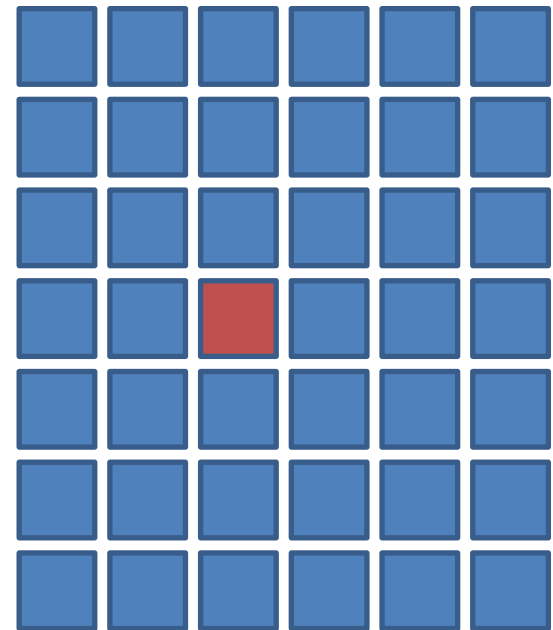
$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

Input



Feature Map



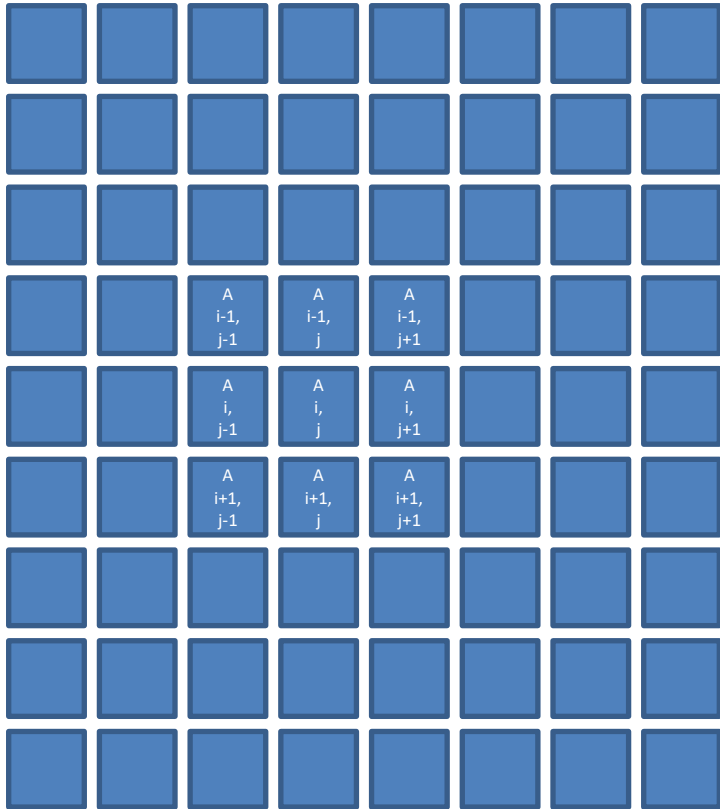
Kernel



$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

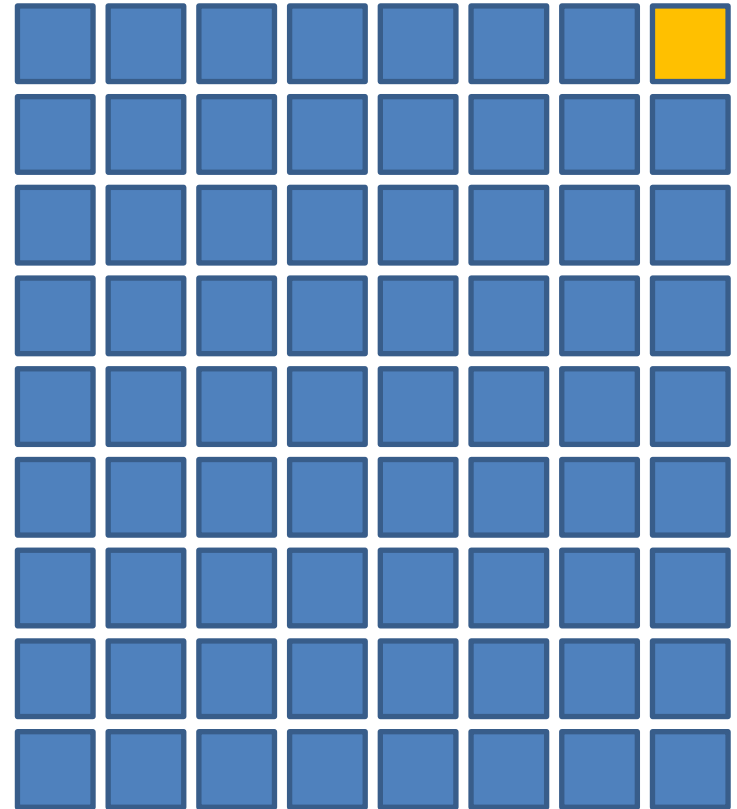
Input



Kernel



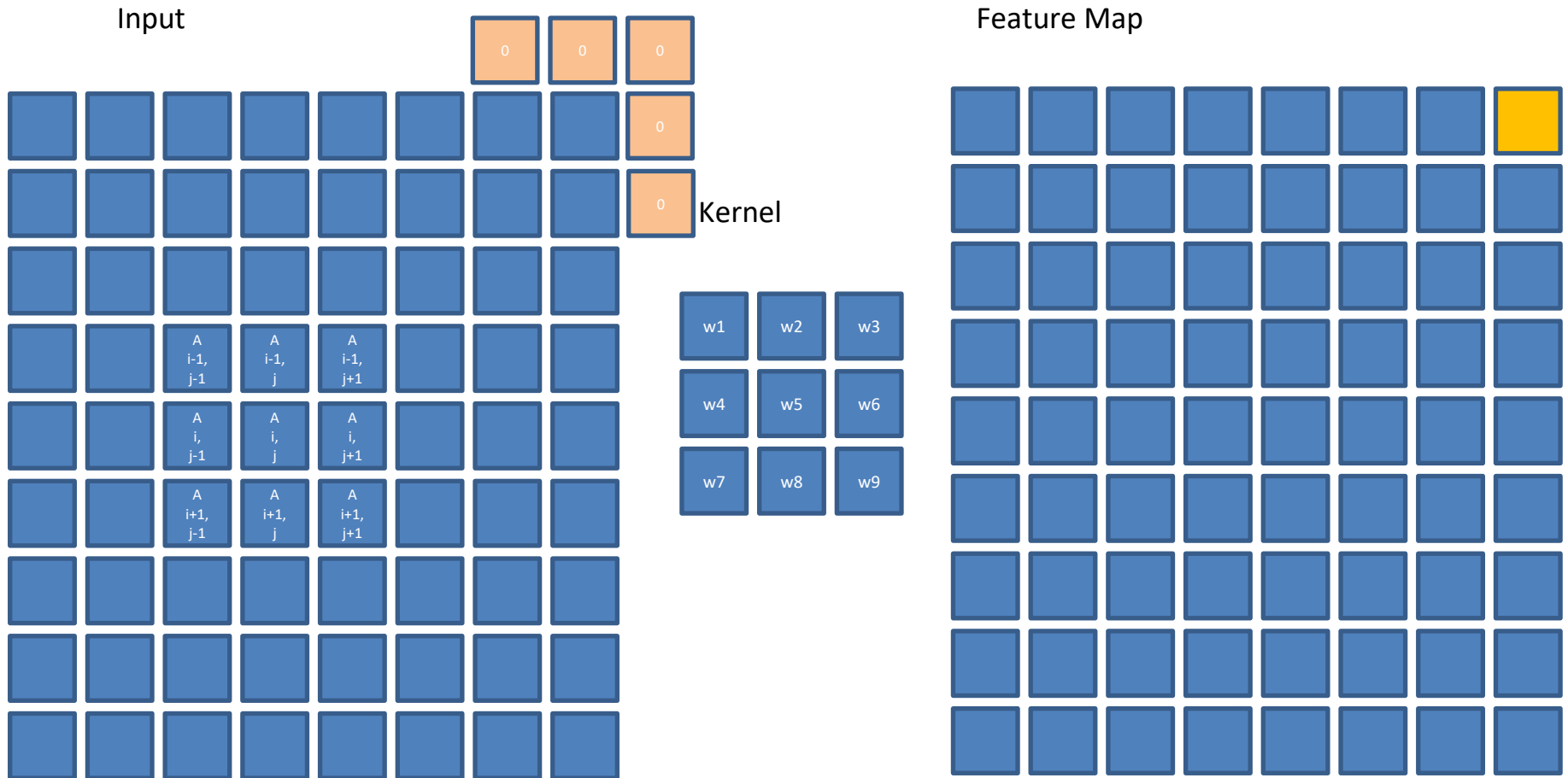
Feature Map



$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Padding

Convolutional Network

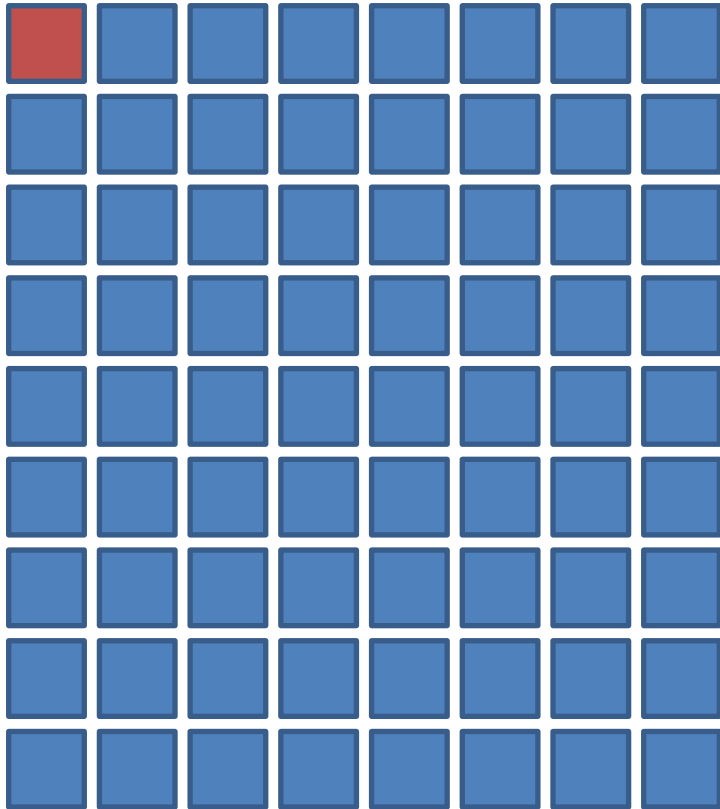


$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Padding

Convolutional Network

Input



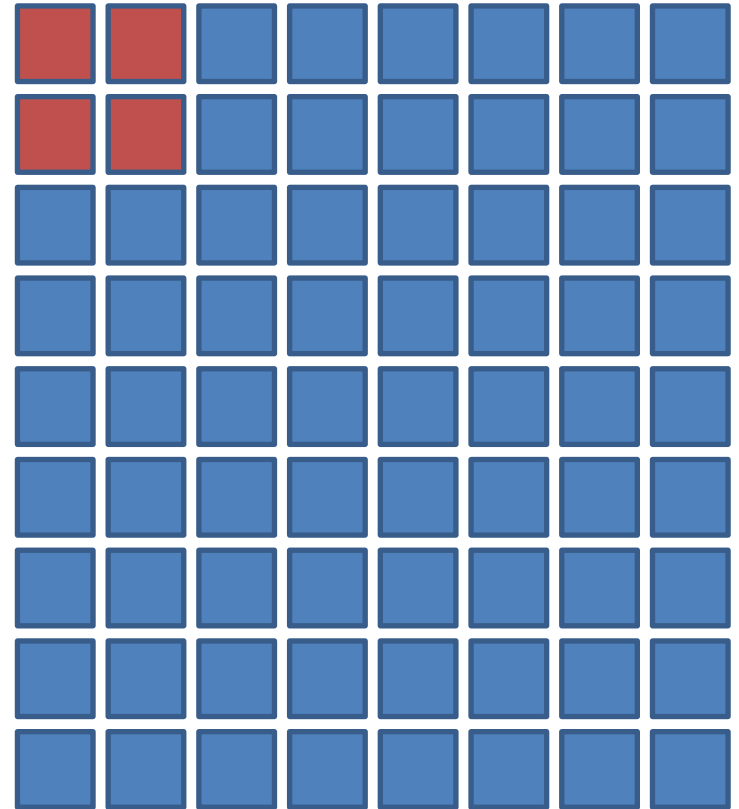
Information
propagation:

1 CNN
layer

Kernel



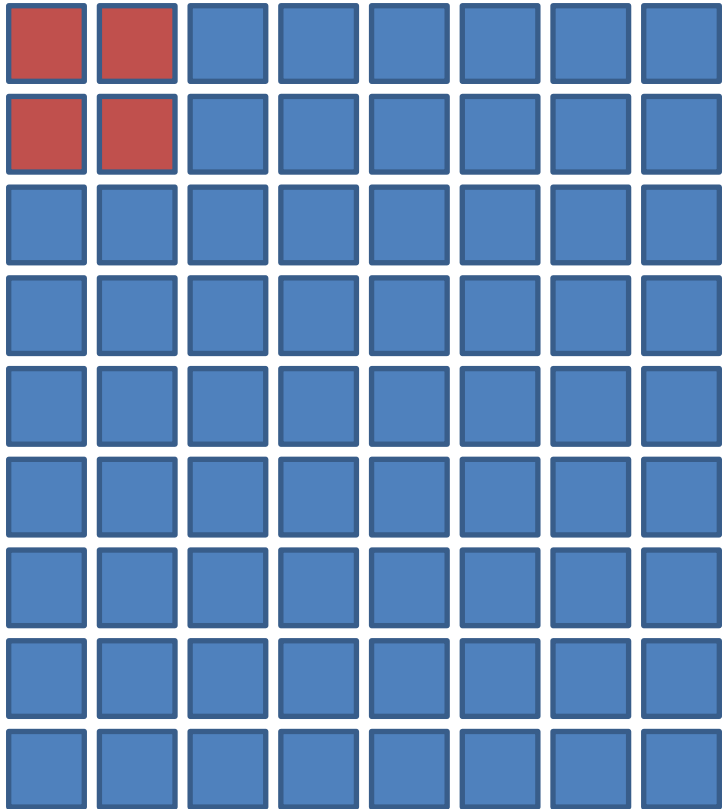
Feature Map



$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

Input



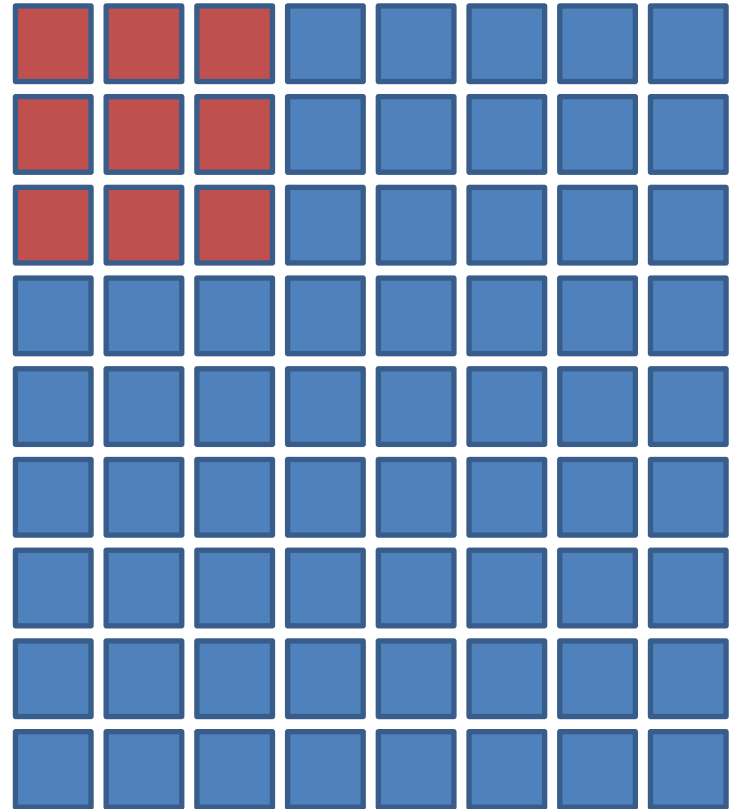
Information
propagation:

2 CNN
layers

Kernel



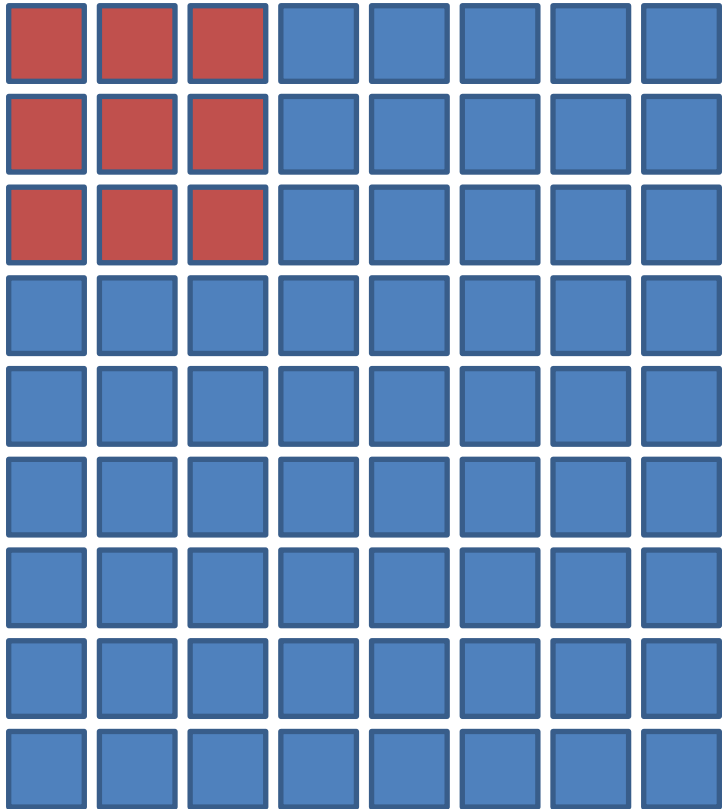
Feature Map



$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

Input



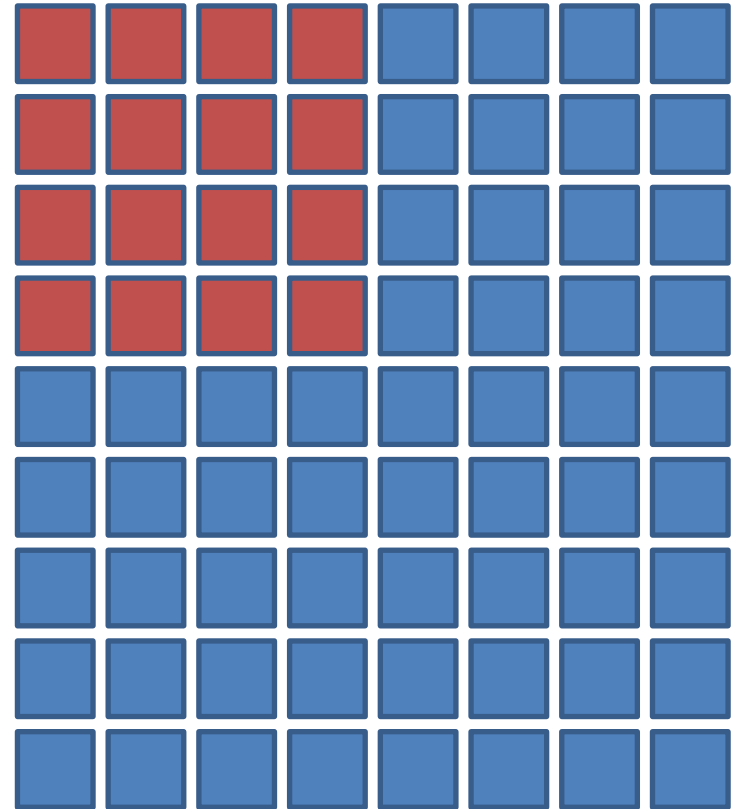
Information
propagation:

3 CNN
layers

Kernel



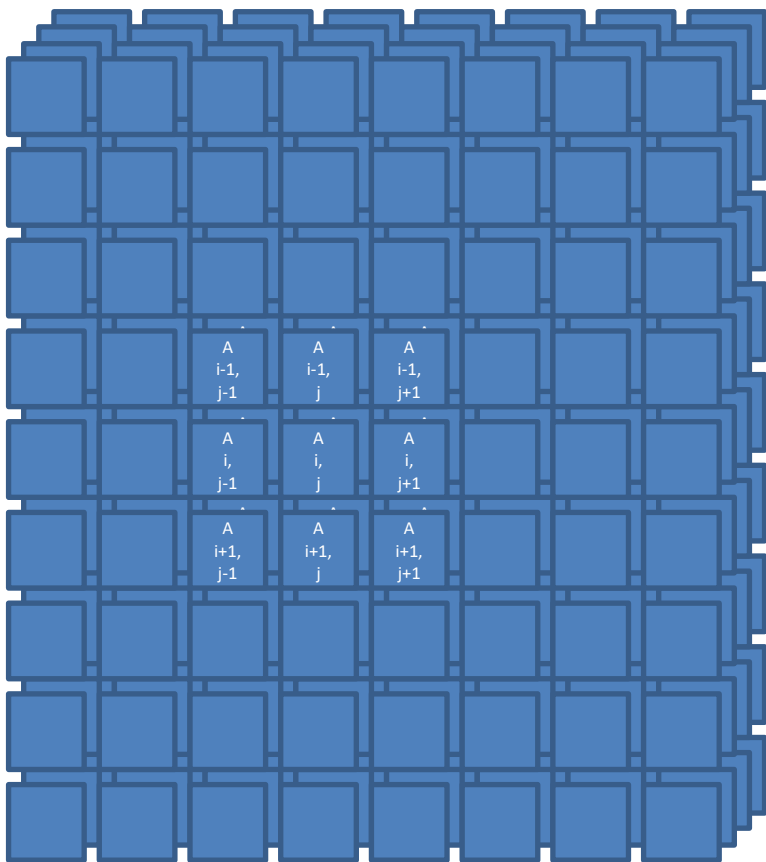
Feature Map



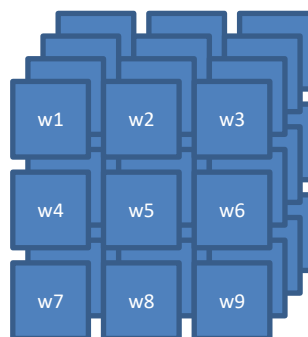
$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} A_{k,l} * w_m$$

Convolutional Network

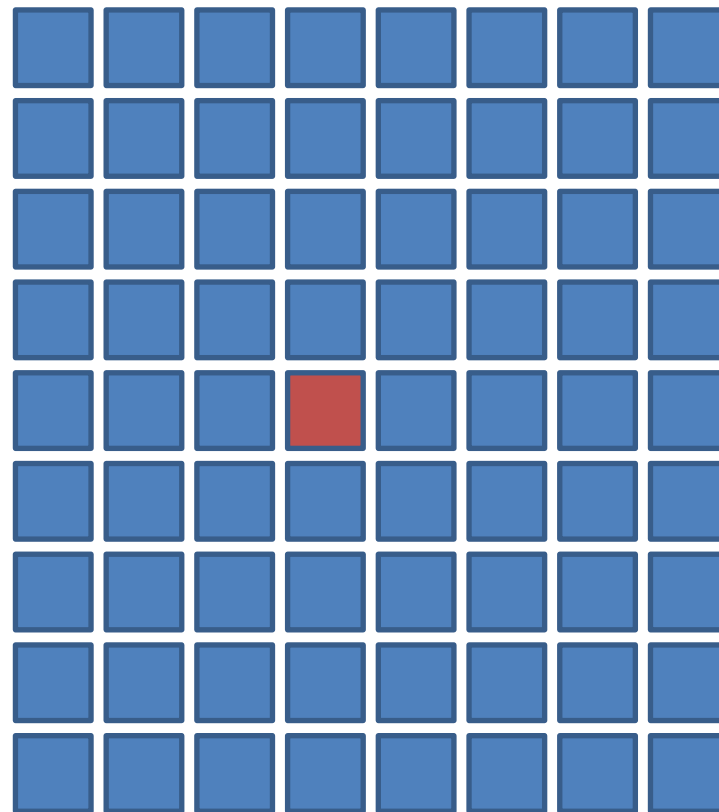
Input



Kernel



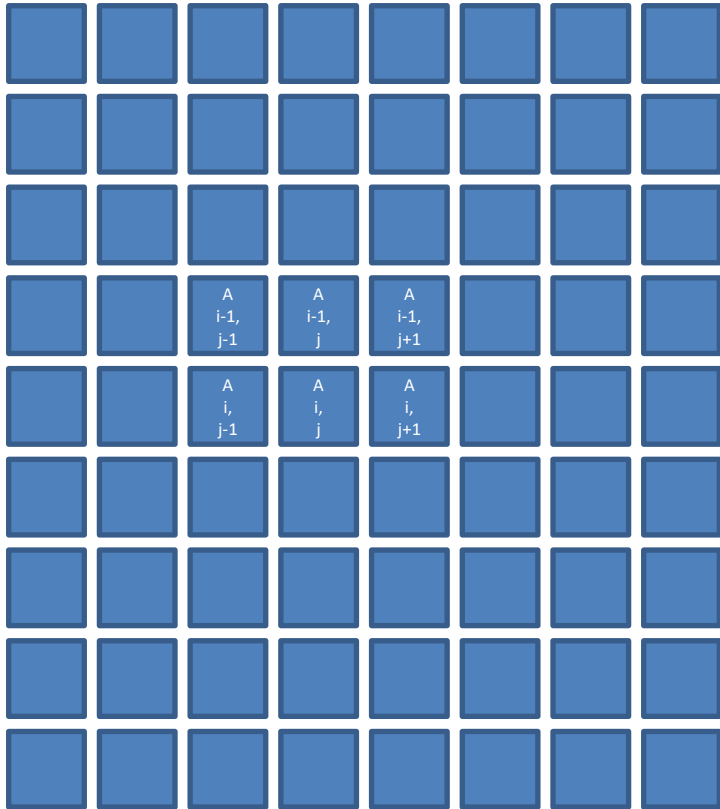
Feature Map



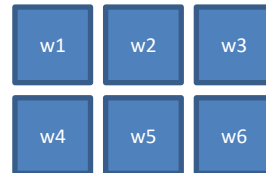
$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} \sum_{m=1}^4 A_{k,l}^m * w_m$$

Convolutional Network

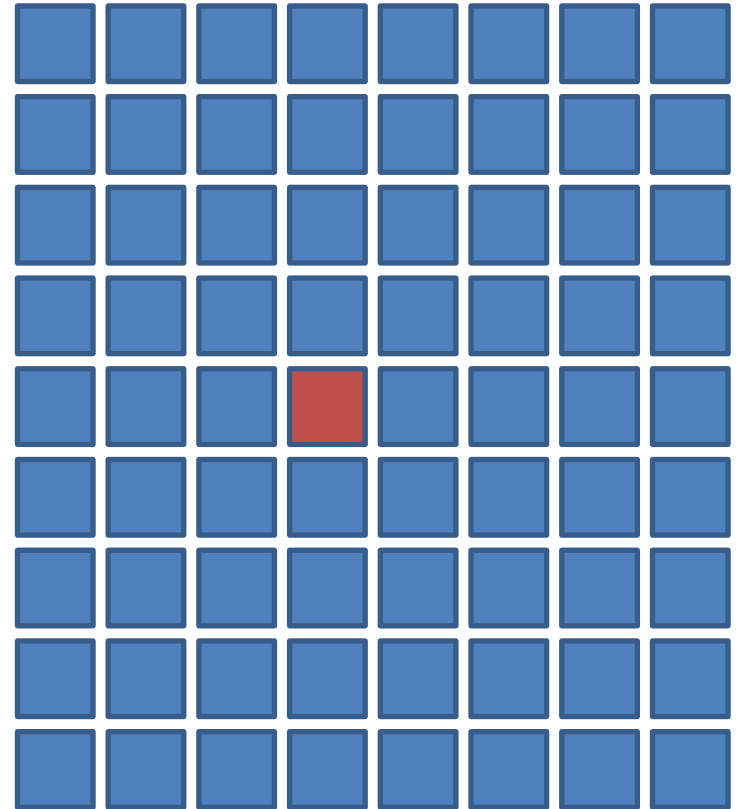
Input



Kernel



Feature Map



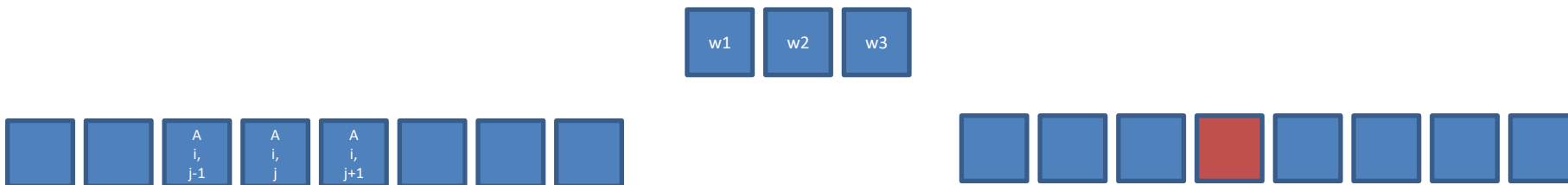
$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^i A_{k,l} * w_m$$

Convolutional Network

Input

Feature Map

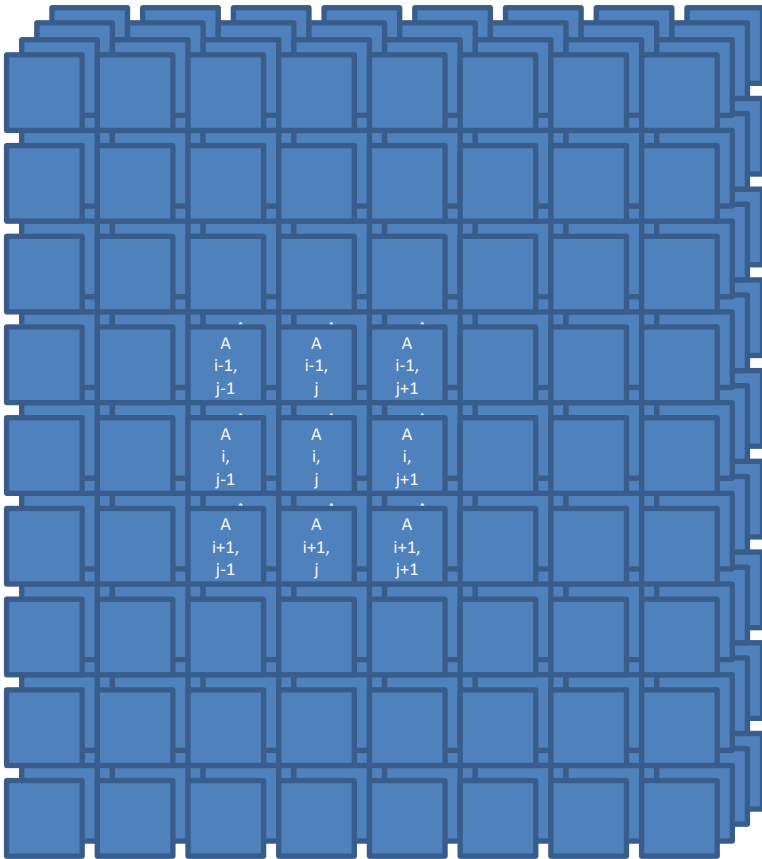
Kernel



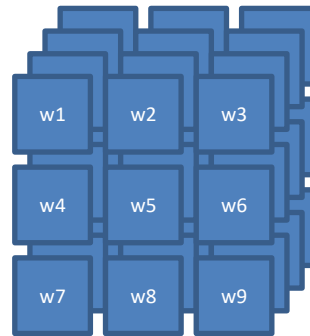
$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i}^i A_{k,l} * w_m$$

Convolutional Network

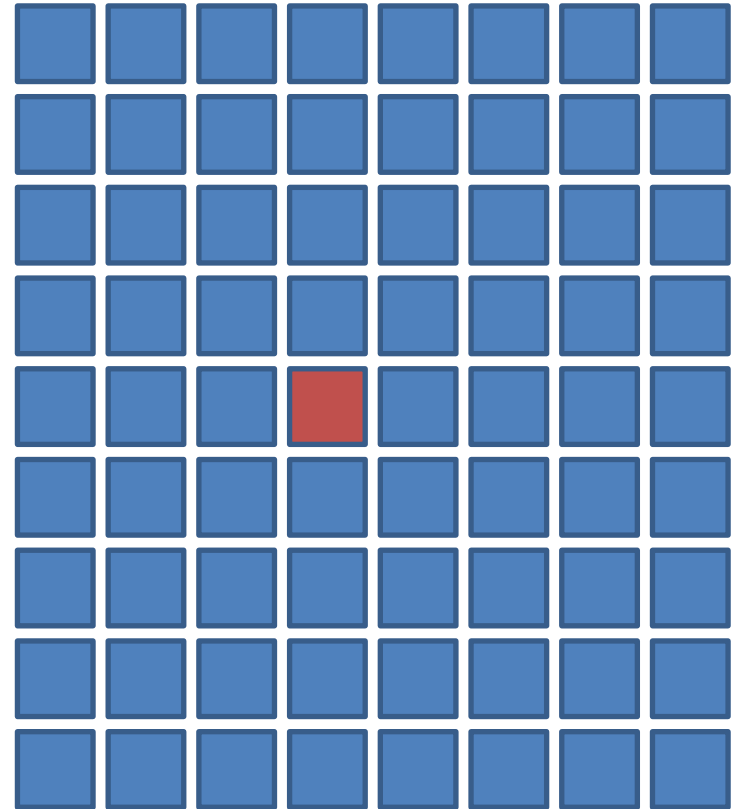
Input



Kernel



Feature Map



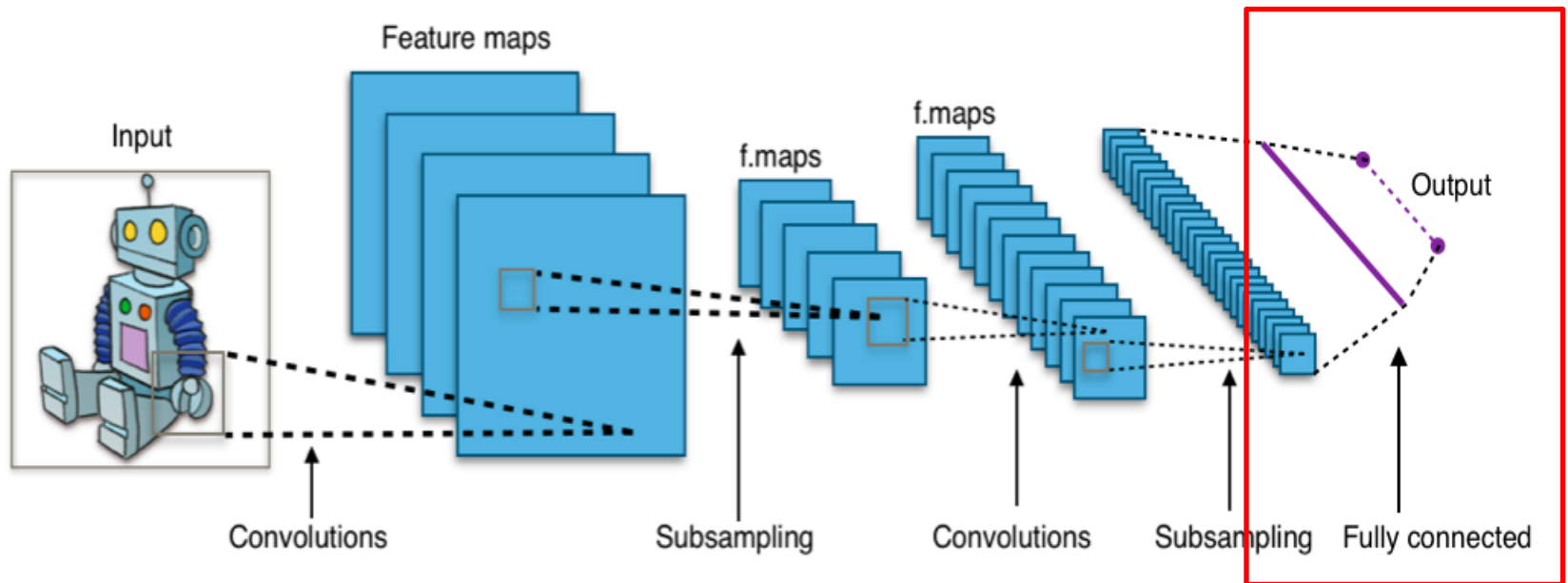
$$f(A_{i,j}) = \sum_{l=j-1}^{j+1} \sum_{k=i-1}^{i+1} \sum_{m=1}^4 A_{k,l}^m * w_m$$

Hyperparameters:

Padding

Convolution filter size

CNN

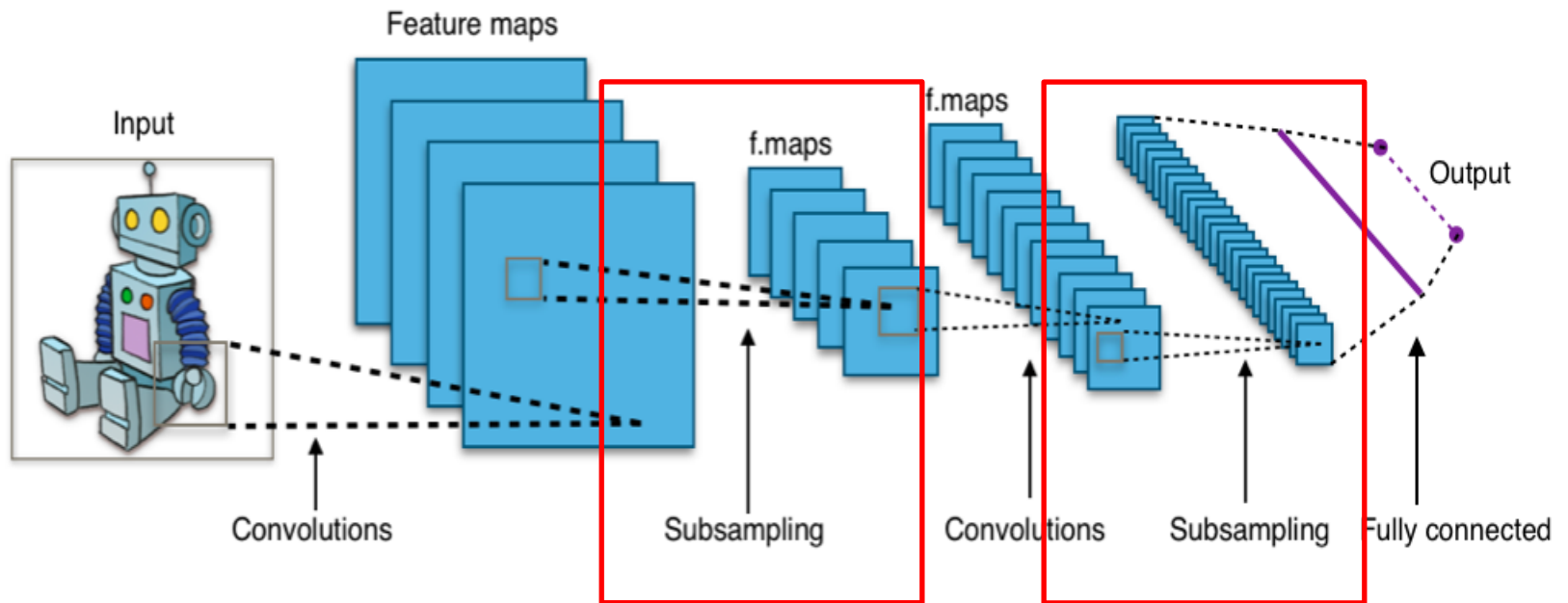


Taking care to flatten the output if needed!!!

Remember: Final layer is critical to your model.
Regression / Classification require different activations

As always, quality of performance depends on training dataset.

Pooling



Pooling

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Feature map



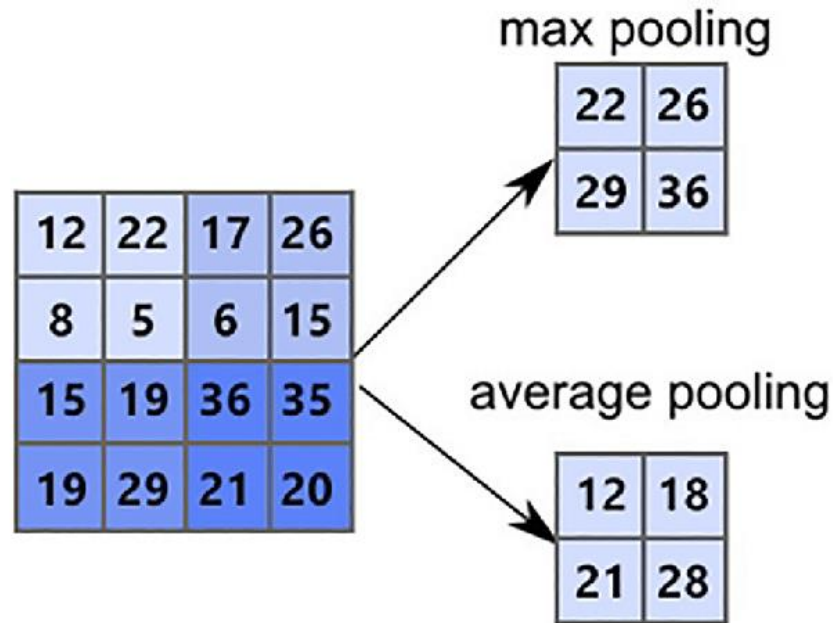
Pooled
Feature map

Effect:

Ability to detect events that were present even when they move slightly, reducing dimension to be able to detect higher level features. Especially usefull with ReLU.

Other poolings option: average (and others but those are the two main ones).

Pooling



Effect:

Ability to detect events that were present even when they move slightly, reducing dimension to be able to detect higher level features. Especially useful with ReLU.

Other poolings option: average (and others but those are the two main ones).

YOLO



You Only Look Once

Identifying objects in an image

YOLO

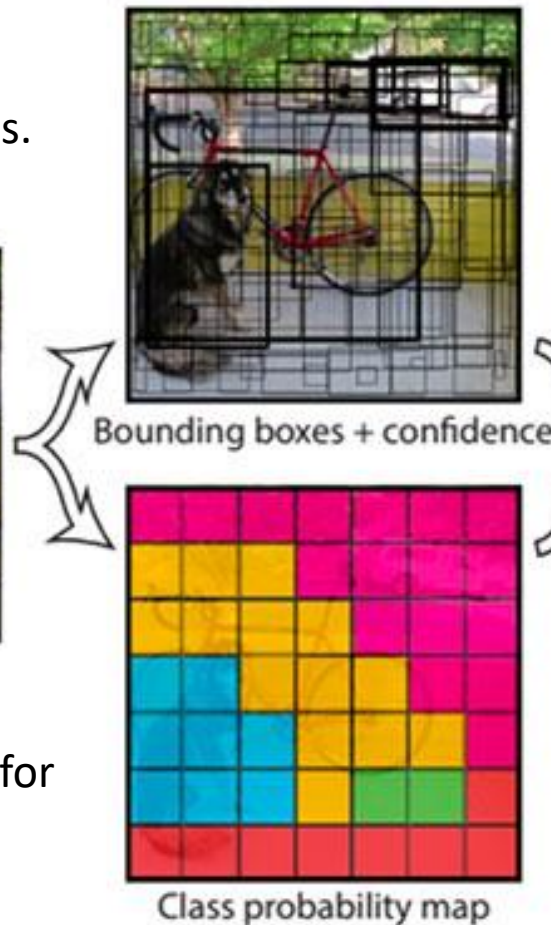
Requires:

- Labeled Training database.
- Importance of potentially having different sized kernels.
- Gridding your input images.



$S \times S$ grid on input

Each grid also tries to predict for each possible label, the probability of being in a given class if there is an object in that grid cell.

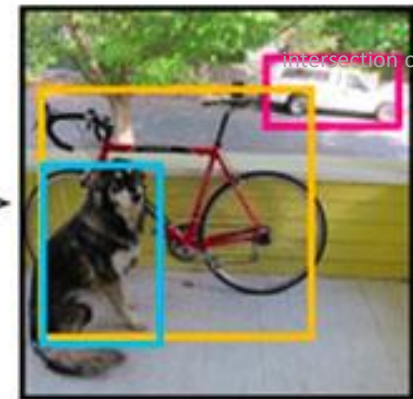


Bounding boxes + confidence

Class probability map

Each grid tries to predict B bounding boxes:

x, y – the center of the bounding box
 w, h – its width and height
the confidence of the prediction (IOU)

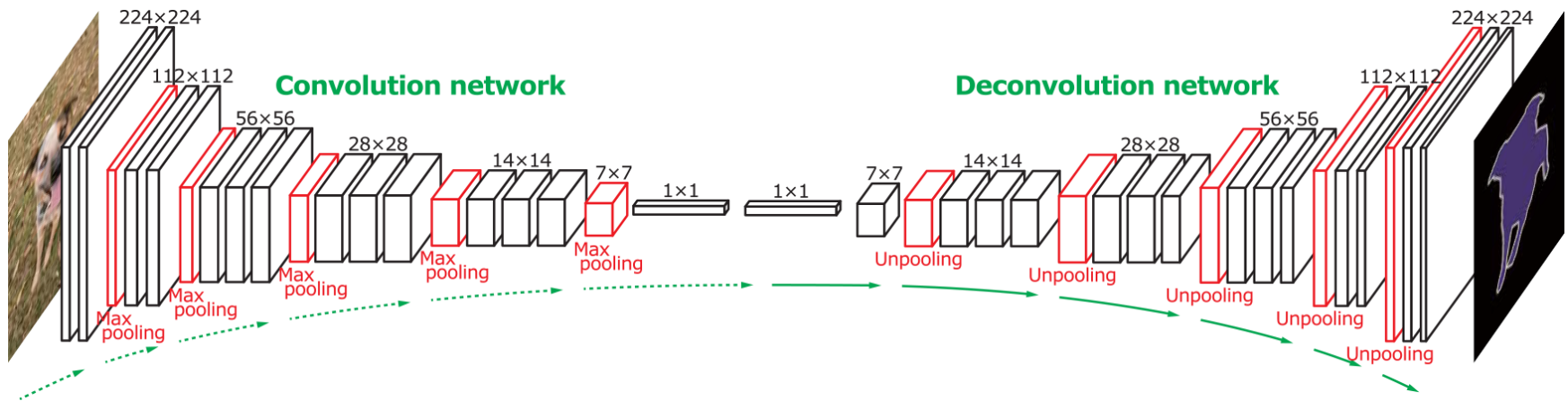


Final detections

Combine all those to detect bounding boxes over a threshold.

Very efficient & does not detect things in background.

CONVOLUTIONAL AUTO ENCODERS



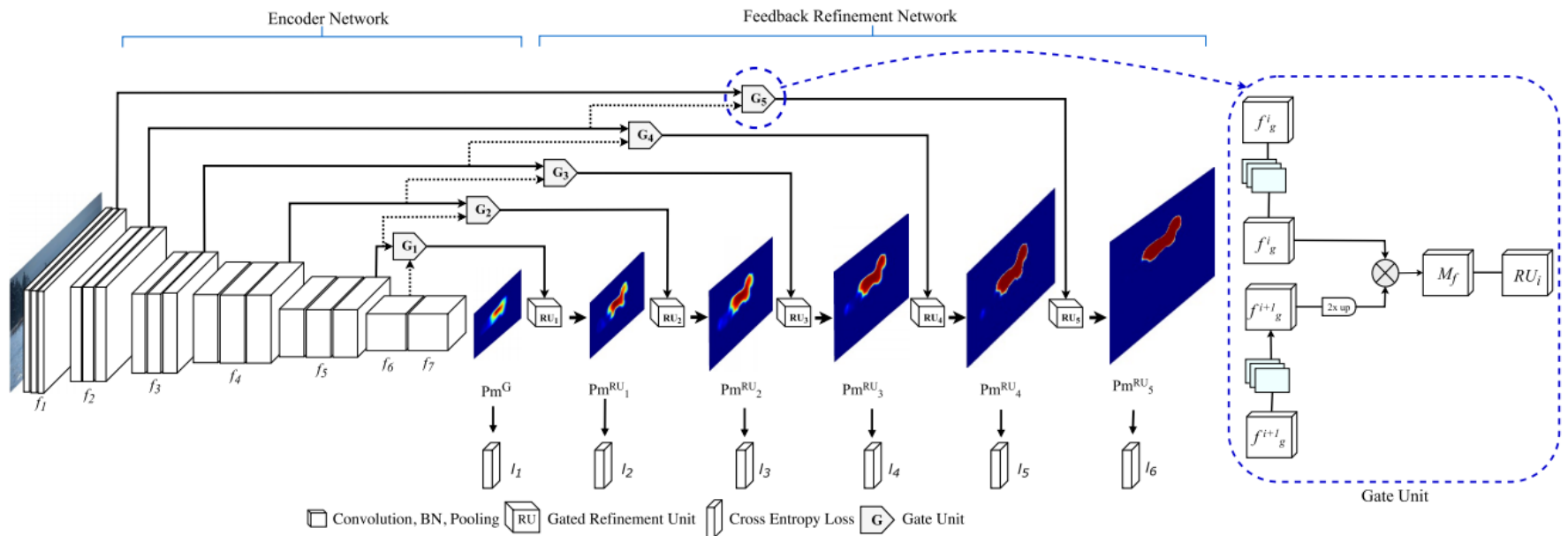
Auto-encoder:

Input = Output

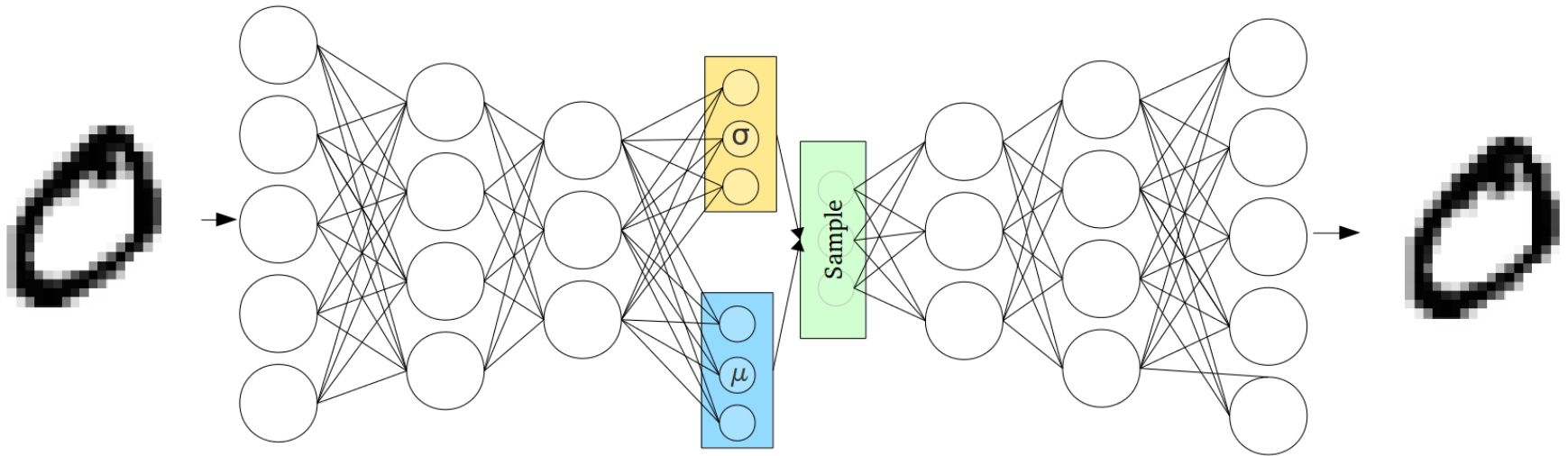
Internal Dimension < than Input Dimension

Skip Connections & Merging layers

Merge layers in Keras.



CONVOLUTIONAL AUTO ENCODERS



Auto-encoder:

Input = Output

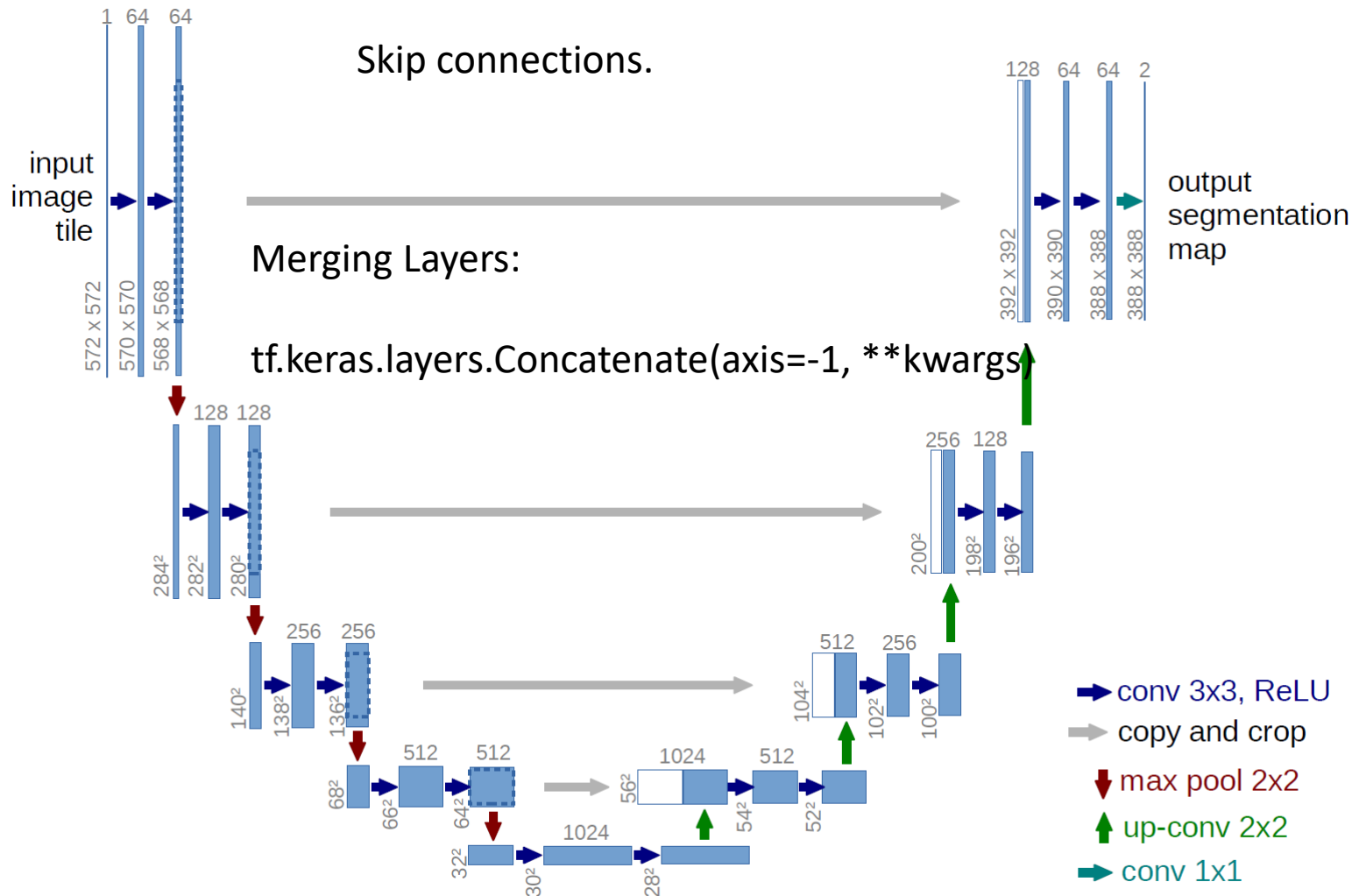
Internal Dimension < than Input Dimension

Variational:

that estimates
use to decode the

In the middle between encoder and decoder a layer
a mean and a sigma, then samples a vector to
image.

U-NET

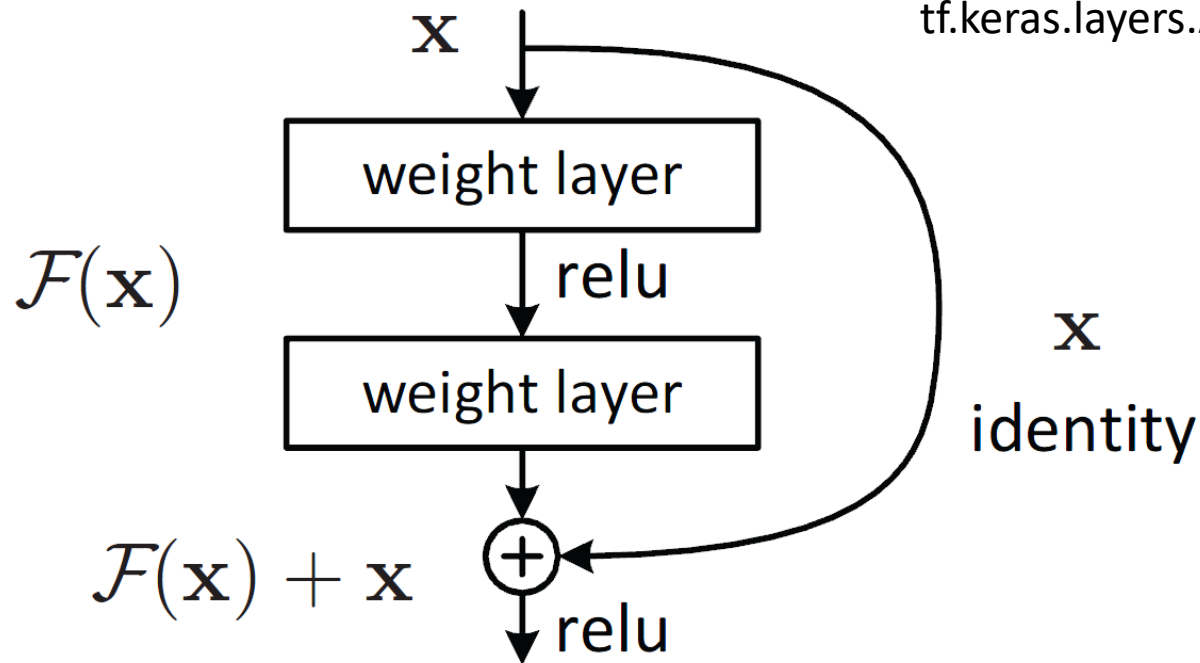


Residual Neural Networks

Empyrical learning of differential equations

Add Layer:

`tf.keras.layers.Add(**kwargs)`



Hyperparameters and their effects

Hyperparameter	Increases capacity when...	Reason	Caveats
Number of hidden units	increased	Increasing the number of hidden units increases the representational capacity of the model.	Increasing the number of hidden units increases both the time and memory cost of essentially every operation on the model.
Learning rate	tuned optimally	An improper learning rate, whether too high or too low, results in a model with low effective capacity due to optimization failure.	
Convolution kernel width	increased	Increasing the kernel width increases the number of parameters in the model.	A wider kernel results in a narrower output dimension, reducing model capacity unless you use implicit zero padding to reduce this effect. Wider kernels require more memory for parameter storage and increase runtime, but a narrower output reduces memory cost.
Implicit zero padding	increased	Adding implicit zeros before convolution keeps the representation size large.	Increases time and memory cost of most operations.
Weight decay coefficient	decreased	Decreasing the weight decay coefficient frees the model parameters to become larger.	
Dropout rate	decreased	Dropping units less often gives the units more opportunities to “conspire” with each other to fit the training set.	

Searching for hyperparameters

Gridsearch: all combinations defined

Randomsearch: requires marginal distributions for each

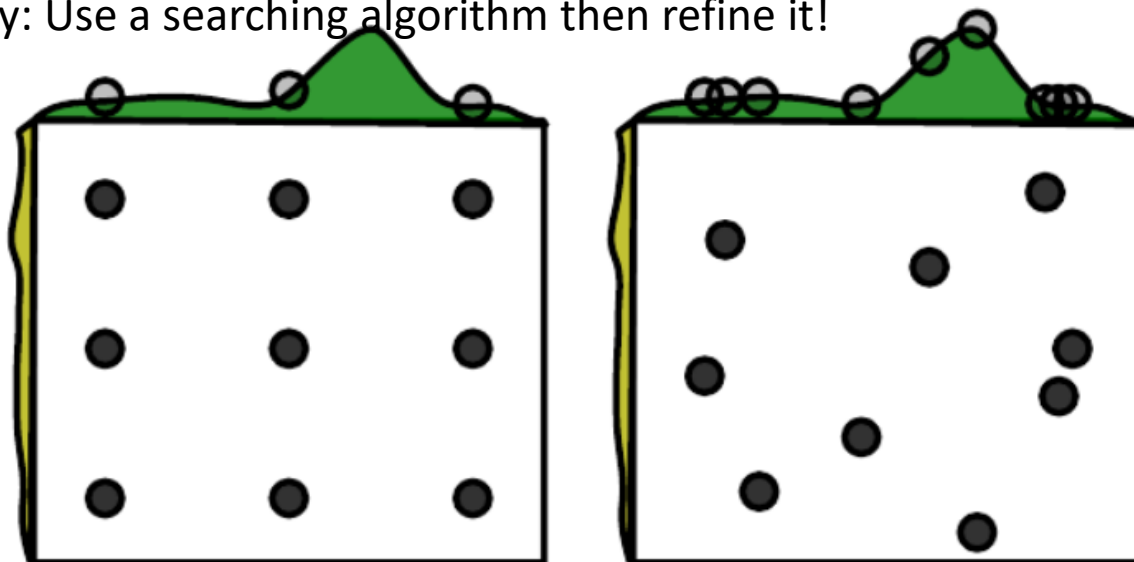
hyperparameter

$$\log_learning_rate \sim u(-1, -5),$$

$$learning_rate = 10^{\log_learning_rate}$$

Example:

Generally: Use a searching algorithm then refine it!

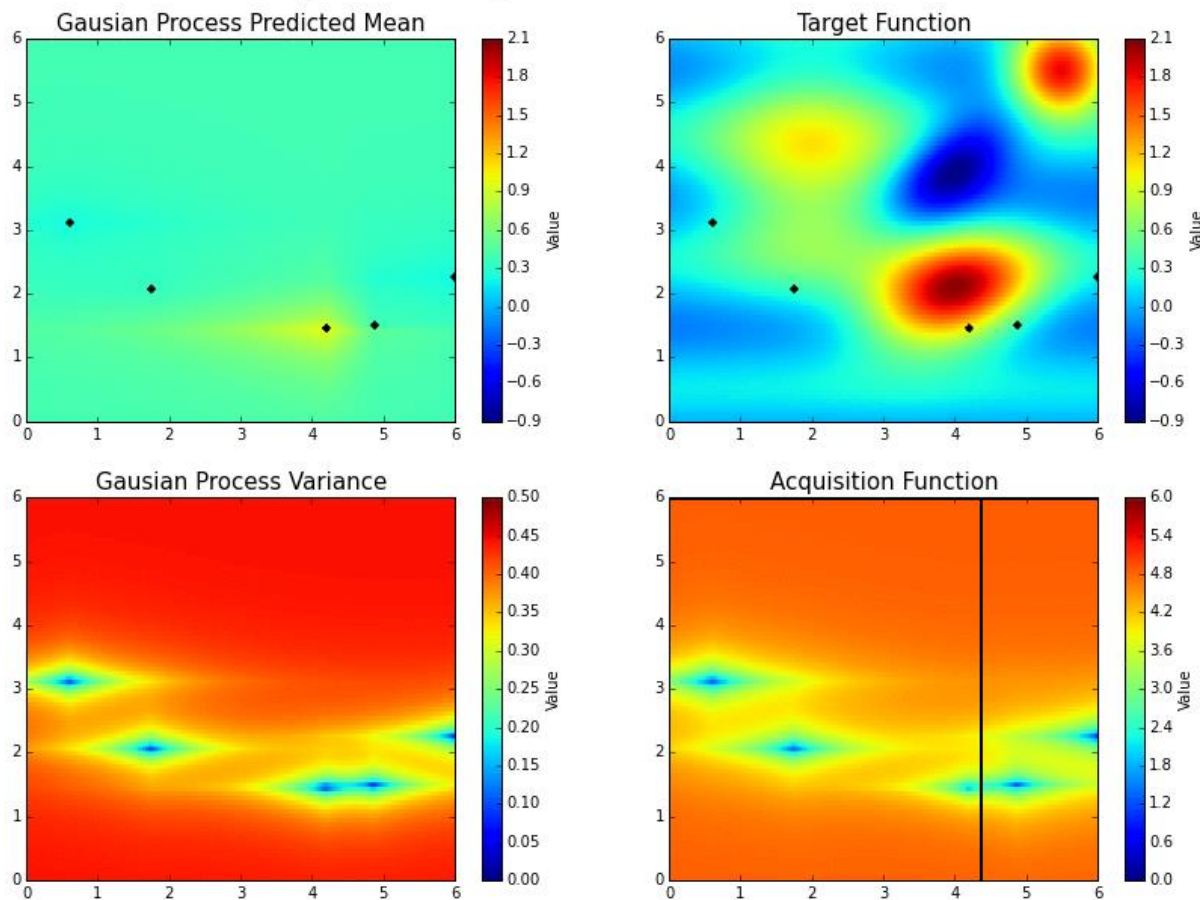


Grid

Random

Searching for hyperparameters

Bayesian Optimization in Action



<https://github.com/fmfn/BayesianOptimization>

Bayesian Optimization: a smarter solution, a harder implementation
Nowadays: hyperopt and other hyperparameter optimization packages
help with that

Optimizers

Usually, the stochastic gradient descent method can be blocked by local minima of the cost function with respect to the training batches.

Momentum: a term that forces our Stochastic Gradient Descent to keep in memory its past trajectory.

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α

Require: Initial parameter θ , initial velocity v

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{x^{(1)}, \dots, x^{(m)}\}$ with corresponding targets $y^{(i)}$.

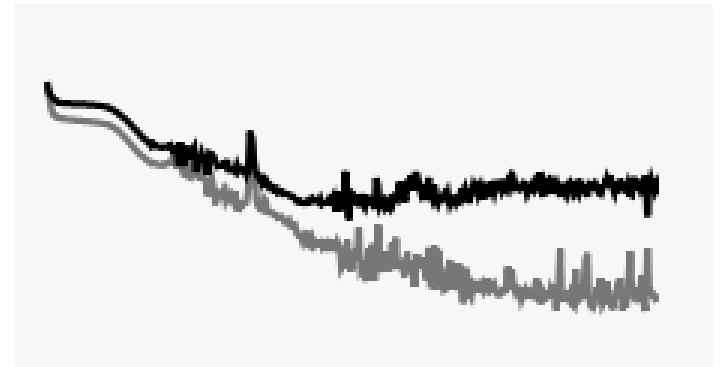
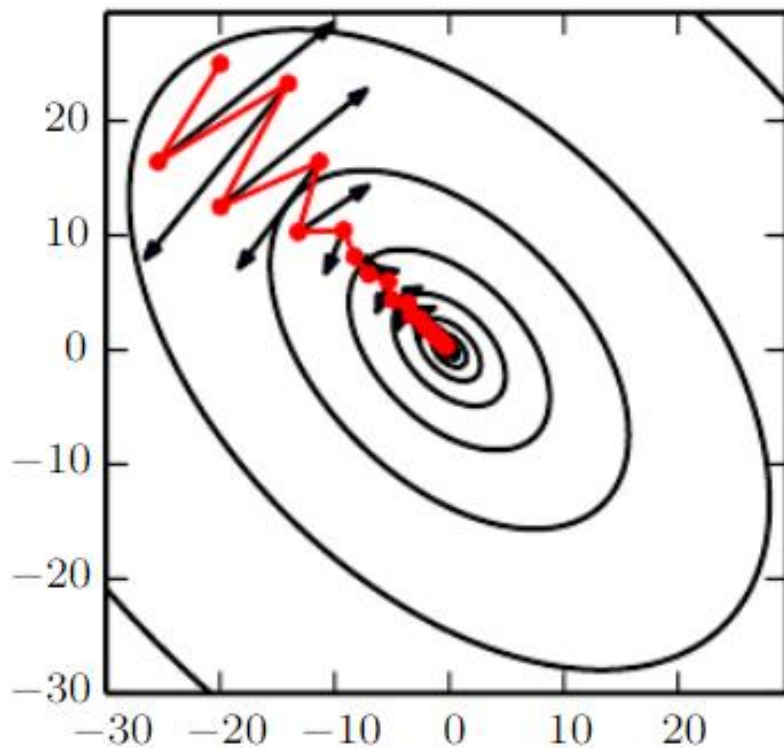
 Compute gradient estimate: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$.

 Compute velocity update: $v \leftarrow \alpha v - \epsilon g$.

 Apply update: $\theta \leftarrow \theta + v$.

end while

Optimizers



Arrows:
SDG without momentum.

Optimizers

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

 Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

 Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

 Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

 Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

 Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

PRACTICAL

<https://cs231n.github.io/convolutional-networks/>

A much more
detailed look at
CNNs